https://crad.ict.ac.cn
x(x): xx−xx, xxxx

大语言模型推理优化方法综述

叶旭明[1,2] 周 健[1,2] 柏澍晗[1,2] 刘 源[3] 朱国平[3] 刘年超[3] 吴 非[1,2]

[1]（华中科技大学武汉光电国家研究中心 武汉 430074）

[2]（华中科技大学计算机科学与技术学院 武汉 430074）

[3]（中电云计算技术有限公司 武汉 430100）

（xumingye@hust.edu.cn）

Survey on Inference Optimization for Large Language Models

Ye Xuming[1,2], Zhou Jian[1,2], Bai Shuhan[1,2], Liu Yuan[3], Zhu Guoping[3], Liu Nianchao[3], and Wu Fei[1,2]

[1]（*Wuhan National Laboratory for Optoelectronics*, *Huazhong University of Science and Technology*, *Wuhan* 430074）

[2]（*School of Computer Science and Technology*, *Huazhong University of Science and Technology*, *Wuhan* 430074）

[3]（*CECloud Computing Technology Co.*, *Ltd.*, *Wuhan* 430100）

Abstract Large language models (LLMs) have attracted significant attention for their exceptional reasoning and long-context capabilities. However, with the growth in model parameter scales and increasingly complex application scenarios, LLMs inference faces severe challenges regarding resource utilization and efficiency. On one hand, the massive parameter scales and the nature of autoregressive generation necessitate the optimization of computation, memory, and communication resource allocation to maximize hardware resource utilization. On the other hand, real-world online serving scenarios introduce highly dynamic concurrent requests and variable computational loads, making it difficult to balance the dual service quality requirements of high throughput and low latency. This paper provides a systematic survey of research progress in LLMs inference optimization, analyzing the design principles and efficacy of existing techniques across three dimensions: Model, Algorithm, and Service. The Model layer significantly reduces resource overhead while maintaining model precision through quantization, sparsification, and model architecture optimization. The Algorithm layer focuses on reconstructing computational logic, breaking performance bottlenecks associated with memory bandwidth and serial generation by optimizing data flow and operator execution patterns. The Service layer concentrates on system-level resource orchestration, achieving Pareto-optimal trade-offs between throughput and latency under specific workload conditions through dynamic request scheduling, architectural disaggregation, and heterogeneous collaboration. Furthermore, we examine the limitations and applicability boundaries of existing techniques, analyze the synergies and conflicts among different optimization paths, and finally discuss future research directions for LLMs inference optimization.

Key words large language models; inference engines; decoding acceleration; memory optimization; model compression

收稿日期: 2026-02-01; **修回日期**: 2026-08-07

基金项目: 工信部高质量发展专项（2540STC62590）; 国家自然科学基金项目（62372197, UA22A2071）; 山东省自然科学基金重点基金项目（ZR2024LZH004）; 深圳市自然科学基金项目（JCYJ20250604190909013）

This work was supported by the High-Quality Development Project of the Ministry of Industry and Information Technology (2540STC62590), the National Natural Science Foundation of China (62372197, UA22A2071), the Key Program of the Natural Science Foundation of Shandong Province (ZR2024LZH004) and the Natural Science Foundation of Shenzhen (JCYJ20250604190909013).

通信作者: 吴非（wufei@hust.edu.cn）

摘 要 大语言模型凭借卓越的逻辑推理和长上下文理解能力吸引了广泛关注。然而,随着模型参数规模的增长和应用场景的日益复杂,大语言模型推理系统面临着严峻的资源与效率挑战。一方面,庞大的模型参数规模与自回归生成特性要求推理系统优化计算、显存和通信资源的分配,提高硬件资源利用率。另一方面,在线服务场景引入了动态变化的并发请求与计算负载,难以兼顾高吞吐与低延迟的服务质量要求。系统性地综述了大语言模型推理优化的研究进展,从模型层、算法层和服务层3个维度分析了现有方法的设计原理与效果。模型层通过量化、稀疏化以及模型架构优化,在保持模型精度的前提下显著降低了资源开销。算法层聚焦于计算逻辑的重构,通过优化数据流转与算子执行模式,打破显存带宽与串行生成的性能瓶颈。服务层则聚焦于系统级资源编排,通过动态请求调度、架构解耦与异构协同,在特定负载条件下实现了吞吐量与延迟的帕累托优化。此外,梳理了各方法的局限性与适用边界,分析了不同优化路径之间的协同与制约关系,最后展望了大语言模型推理优化的未来研究方向。

关键词 大语言模型;推理引擎;解码加速;显存优化;模型压缩

中图法分类号 TP311.13; TP30

DOI: 10.7544/issn1000-1239.202660068 **CSTR:** 32373.14.issn1000-1239.202660068

基于Transformer[1]架构的大语言模型(large language models, LLMs)是当前深度学习研究的主流方向,以下简称大模型。随着训练数据的不断扩充以及模型参数的快速增长,大模型所具备的复杂逻辑推理能力和长上下文理解能力显著增强[2-4],在聊天机器人[5]、逻辑推理[6]、代码生成[7]等多个领域都展现出了巨大潜力和应用前景。例如,以ChatGPT[8]为代表的对话系统改变了获取信息的方式,Github Copilot[9]等代码助手显著提升了编码效率。

然而,大模型在实际部署中的推理效率仍然面临着许多亟待解决的问题,引起了学术界和工业界的广泛关注。首先,大模型推理的自回归性质对计算和显存带宽资源提出了不同要求。不同于传统深度学习模型的单次前向传播,大模型推理过程分为预填充和解码2个显著不同的阶段[10]。在预填充阶段需要对用户输入进行并行处理,属于计算密集型任务。解码阶段则需逐个生成词元,计算并行度受限,但需要频繁访问键值缓存(key value cache, KV cache),属于访存密集型任务。这种差异性导致推理过程中GPU利用率低下。其次,目前模型规模通常超出了单卡显存上限。以Llama-3 70B为例[11],在FP16精度下需要约140 GB显存,DeepSeek-V3的显存占用达到了1 370 GB[12]。这不仅导致模型推理所需的计算与显存资源显著增加,而且实际推理过程中需要采用多卡甚至多机部署,带来了额外的通信开销,进一步限制了推理速度的提升。最后,随着智能体[13]等复杂应用的兴起,多轮交互与自主规划机制导致单任务的推理链路显著延长,极大地提高了业务对低延迟的敏感度。如何在保障模型生成质量的条件下实现低延迟、高吞吐的推理服务成为大模型应用落地的首要难题。

在此背景下,研究者们聚焦于大模型推理加速这一重要方向,在维持模型推理性能的基础上,降低推理延迟和部署成本。近几年来,业界与学术界涌现出了一系列包含模型压缩、算子优化、服务调度等不同维度的大模型推理优化方法,这些方法在生成质量、应用场景、资源需求等方面呈现出不同的权衡特征,展示了大模型推理过程的巨大优化空间。如图1所示,本文从模型层、算法层、服务层3个角度对已有技术进行分类,分类依据是优化手段的性质而非优化对象的形态:模型层改变模型参数或推理中间状态的数值表示与结构形态,核心在于张量自身的如何表示这一维度;算法层改变推理计算逻辑与数据流的组织方式,聚焦于如何计算这一维度;服务层优化则改变系统资源的编排方式,作用于如何调度这一维度。

在模型层,核心挑战在于庞大的模型参数规模带来的巨额计算与存储开销。现有优化方法可以分为量化、稀疏化以及高效架构演进3个维度。其中量化方法[14]减少了显存与带宽需求,本文详细分析了权重和激活值量化方案,以及推理过程中动态增长的键值缓存的量化,阐述不同量化粒度在压缩率与模型性能之间的权衡。稀疏化[15]进一步探索了模型内部的冗余性,包括静态稀疏化和动态稀疏化,旨在通过减少激活参数量以降低计算开销。模型架构优化则关注于注意力机制计算复杂度的2次增长问题:一方面讨论了MQA(multi-query attention)、GQA(group-query attention)、MLA(multi-head latent attention)[16-18]三种注

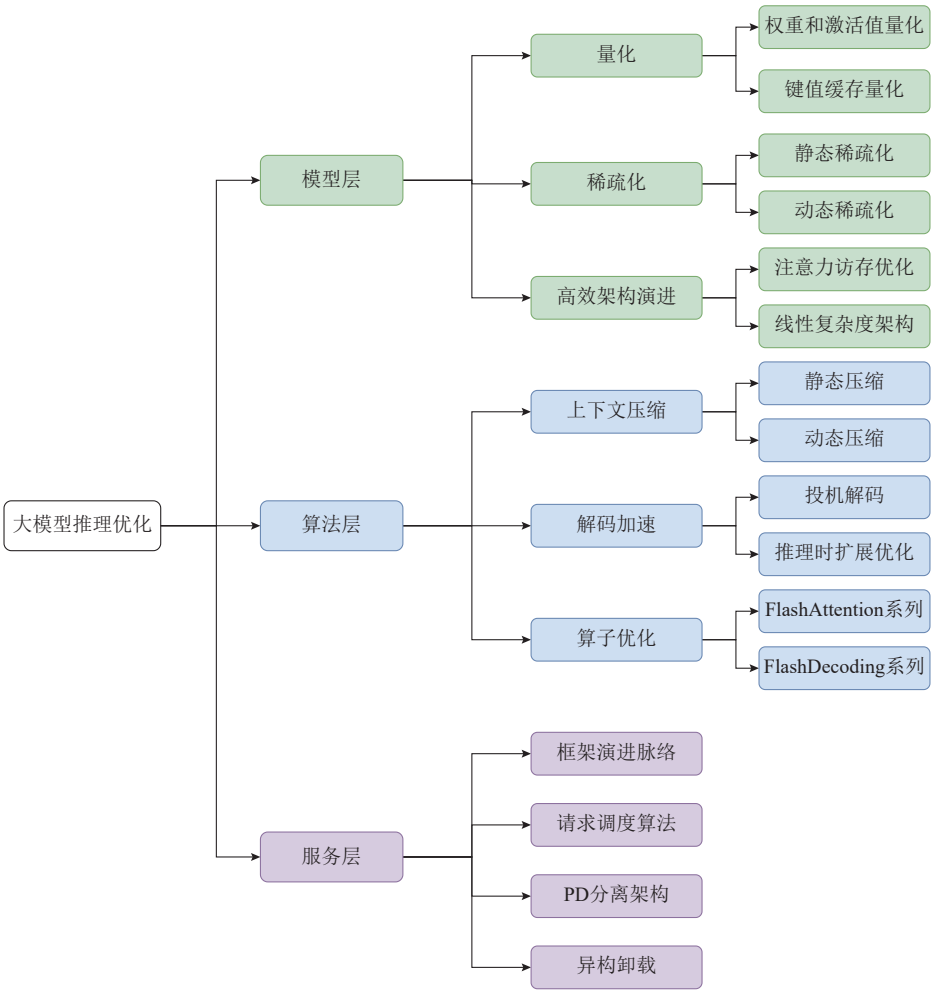


Fig. 1 Taxonomy of inference optimization techniques for LLMs

图 1 大模型推理优化技术分类

意力变体，通过优化访存模式提高硬件利用率；另一方面讨论了替代 Transformer 范式的线性复杂度架构[19-20]，探讨其在长序列推理中的计算效率。

算法层聚焦于计算逻辑的优化，旨在通过优化数据流和计算模式以突破推理性能瓶颈。首先，针对长序列推理带来的显存压力，本文探讨了上下文压缩方法[21]，通过输入侧的静态筛选与运行时的动态键值缓存驱逐策略，显著降低了显存开销；其次，针对解码过程中大模型自回归生成的特性，本文分析了投机解码[22]和推理时计算扩展优化[23]2 种解码加速策略，在保证精度的同时显著提升了推理吞吐量；最后，面对高并发场景下的显存带宽瓶颈，本文梳理了以 FlashAttention[24]和 FlashDecoding[25]为代表的算子优化方法，实现了 GPU 利用率的极致优化。

服务层从推理服务的角度出发，研究如何在高并发、动态负载以及异构计算等复杂场景下，通过系统级的资源编排实现吞吐量、延迟与服务质量的平衡。针对这一挑战，业界提出了一系列推理引擎，这些引擎通过重构显存管理与执行流，奠定了高性能推理服务的基础[26-32]。本文首先梳理了目前常用推理引擎的演进脉络，然后主要围绕请求调度、预填充与解码分离架构和异构卸载 3 个关键内容展开。其中请求调度算法通过连续批处理与分块预填充等策略，解决了高并发场景下的队头阻塞与负载不均；预填充与解码分离架构通过解耦部署与流水线并行，消除了 2 个阶段在计算与显存上的资源争抢；异构卸载方法则致力于突破单卡显存的物理限制，利用系统内存与外部存储实现低成本的显存扩展，并支持将计算任务卸载至 CPU 协同执行，从而提高硬件资源利用率。

接下来，本文将首先介绍大模型推理背景与挑战，再依次分析和总结这些推理优化方法。

1 大模型推理背景与挑战

本节首先介绍了 Transformer 架构，剖析其内部

组件与计算逻辑。接着阐述了大模型特有的自回归推理机制,分别对预填充和解码2个阶段在计算密度与访存行为上的根本差异进行分析。最后,结合模型自身的架构特征以及日益复杂的实际应用需求,总结了大模型高效部署面临的核心挑战。

1.1 Transformer 架构

在深度学习领域,Vaswani 等人[1]提出 Transformer 架构,摒弃了循环结构的顺序依赖,通过多头自注意力机制(multi-head attention, MHA)实现了全局上下文的并行建模,已成为大模型的事实标准架构。Transformer 由编码器(encoder)和解码器(decoder)组成,它们又分别由多个 Transformer 块堆叠形成。目前主流的大模型均采用 Decoder-Only 模型,即仅保留了其中的解码器部分。如图 2 所示,大模型的模型架构包含了 1 个嵌入层、若干个 Transformer 块、1 个输出层:

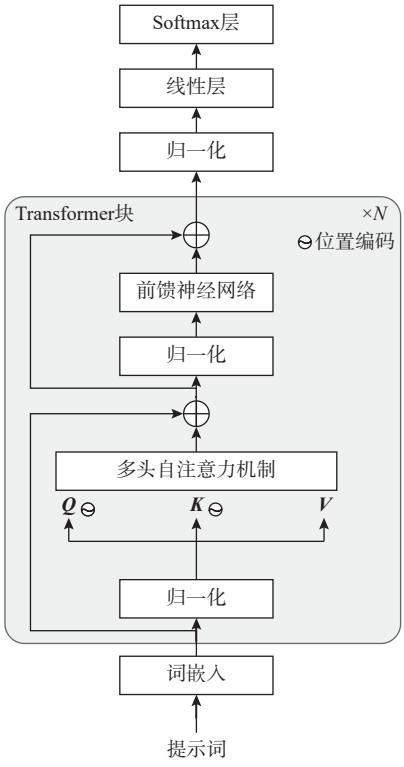


Fig. 2 Illustration of the Decoder-Only model architecture[11]

图 2 Decoder-Only 模型架构示意图[11]

1)嵌入层。用户输入提示词后,分词器将其切分为若干词元,每个词元通过嵌入层转换为向量,然后输入给解码器。假设词表大小为 V,隐藏层维度为 d_{model},嵌入矩阵为 $\boldsymbol{W}_{\text{e}} \in \mathbb{R}^{V\times d_{\text{model}}}$。对于第 i 个词元 x_i,其嵌入向量定义为

$$\boldsymbol{h}_i^{(0)} = Embedding(x_i)\,, \tag{1}$$

目前大模型普遍摒弃了绝对位置编码,不再直接相加于嵌入层,而是采用旋转位置编码[33],将位置信息融入后续的注意力计算中。

2)Transformer 块包含 2 个核心子层:多头自注意力机制(multi-head self-attention, MHSA)和前馈神经网络(feed-forward network, FFN),并在每个子层前应用归一化(通常为 RMSNorm)[34]和残差连接。对于第 l 层,输入为所有词元的嵌入向量 $\boldsymbol{H}^{(l-1)}$,具体计算过程如下:

$$\tilde{\boldsymbol{H}}^{(l)} = MHSA\left(RMSNorm\left(\boldsymbol{H}^{(l-1)}\right)\right) + \boldsymbol{H}^{(l-1)}\,, \tag{2}$$

$$\boldsymbol{H}^{(l)} = FFN\left(RMSNorm\left(\tilde{\boldsymbol{H}}^{(l)}\right)\right) + \tilde{\boldsymbol{H}}^{(l)}\,。 \tag{3}$$

式(2)中的多头自注意力机制是提取语义特征的核心使所有词元在上下文窗口内与其他可见词元建立关联,从而有效建模序列内部的长程依赖关系,获取嵌入了上下文语义的向量表征,即隐状态。具体实现上,输入 $\boldsymbol{X}$ 通过投影矩阵 $\boldsymbol{W}_{\boldsymbol{Q}},\boldsymbol{W}_{\boldsymbol{K}},\boldsymbol{W}_{\boldsymbol{V}}$ 映射为查询($\boldsymbol{Q}$)、键($\boldsymbol{K}$)和值($\boldsymbol{V}$)向量。为了引入序列的顺序信息,旋转位置编码被显式地应用于查询向量和键向量上。对于第 h 个注意力头,计算过程如下:

$$\boldsymbol{Q}_h = RoPE\left(\boldsymbol{X}\boldsymbol{W}_{\boldsymbol{Q}}^{(h)}\right)\,, \tag{4}$$

$$\boldsymbol{K}_h = RoPE\left(\boldsymbol{X}\boldsymbol{W}_{\boldsymbol{K}}^{(h)}\right),\ \boldsymbol{V}_h = \boldsymbol{X}\boldsymbol{W}_{\boldsymbol{V}}^{(h)}\,, \tag{5}$$

$$Attention(\boldsymbol{Q}_h,\boldsymbol{K}_h,\boldsymbol{V}_h) = Softmax\left(\frac{\boldsymbol{Q}_h\boldsymbol{K}_h^{\mathrm{T}}}{\sqrt{d_{\text{head}}}} + \boldsymbol{M}\right)\boldsymbol{V}_h\,, \tag{6}$$

其中 $\boldsymbol{M}$ 为因果掩码(causal mask),确保位置 t 只能关注到 t 及其之前的词元。最后将多个头得到的注意力结果堆叠起来,作为下一层的输入。

式(3)前馈神经网络的主要作用是接收注意力层的输出,并对其进行进一步的非线性变换,以捕获更复杂的特征和表示。其通常由 2 层线性变换组成:第 1 层线性变换负责将输入映射到更高维度的空间,并引入非线性(即激活函数,用 ϕ 表示);第 2 层线性变换则负责将输出映射回与输入相同的维度,通常不引入额外的非线性:

$$FFN(\boldsymbol{x}) = \phi\left(\boldsymbol{x}\boldsymbol{W}^{(1)} + \boldsymbol{b}^{(1)}\right)\boldsymbol{W}^{(2)} + \boldsymbol{b}^{(2)}\,。 \tag{7}$$

3)输出层将最后一个词元对应的特征向量映射为下一个词元的概率分布。具体而言,该层首先利用一个线性层将特征向量投影至词表空间,随后通过 *Softmax* 函数对投影结果进行归一化,从而得到下一个词元的预测概率:

$$P(x_{t+1}|(x_1,x_2,\cdots,x_t)) = Softmax(\boldsymbol{h}_t\boldsymbol{W}_{\text{vocab}} + \boldsymbol{b})\,。 \tag{8}$$

1.2 自回归推理范式:预填充与解码

由上述 Transformer 计算过程可知,大模型的推理过程并非单次前向传播完成,而是通过自回归的

方式逐个生成词元。这种生成模式在系统实现层面被进一步划分为 2 个性质截然不同的阶段：预填充（prefill）与解码（decoding）[26]。如图 3 所示，预填充阶段处理用户输入的提示词序列，并输出首个词元，其时间开销记为首词延迟（time to first token, TTFT）。由于此时输入序列完全已知，可以一次性计算出所有词元对应的查询、键和值向量，并通过矩阵-矩阵乘法高效完成注意力运算。注意力机制的计算量随输入长度呈 2 次方增长，属于典型的计算密集型任务，能够充分发挥 GPU 的并行计算能力。

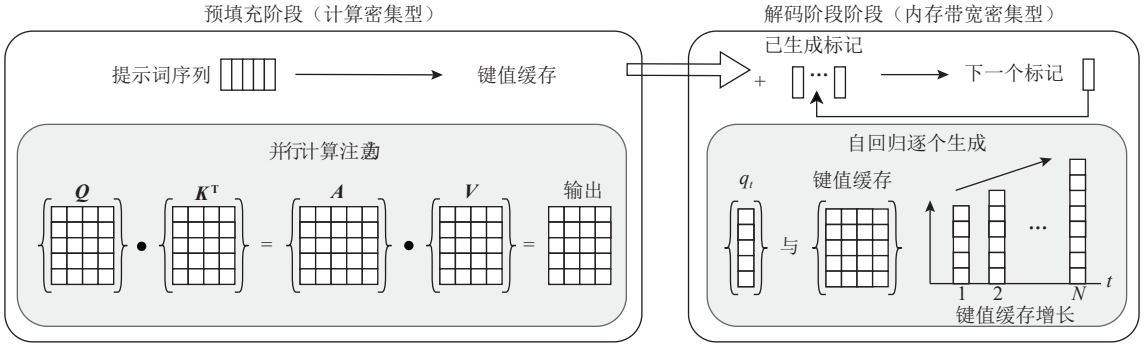


Fig. 3 Comparison of computational characteristics between prefill and decoding phases

图 3 预填充与解码阶段的计算特性对比

为了避免在解码阶段重复计算历史词元的注意力，当前推理引擎普遍引入键值缓存机制。该机制将每步产生的键和值向量存储下来，在后续生成步骤中，系统直接读取缓存即可完成注意力运算。然而，随着生成序列不断变长，键值缓存的显存占用呈线性增长。并且在长上下文场景中，其显存开销甚至远超模型静态权重本身，成为制约显存容量的主要瓶颈。

解码阶段采用自回归模式逐一生成后续词元，其执行效率通常由单词元生成耗时（time per output token, TPOT）来衡量。该阶段每次仅处理新生成的单个词元，构造其对应的查询向量，并读取此前累积的全部键值缓存矩阵以完成注意力计算。由于该过程主要涉及向量-矩阵乘法，单步计算量很小，计算单元长时间处于等待数据加载的状态。因此，解码阶段属于典型的访存密集型任务，其生成速度主要受限于硬件显存带宽，而非计算峰值性能。

从预填充到解码，计算模式发生了根本性的转变，导致 GPU 利用率大幅波动。同时，解码阶段的自回归特性也带来了严峻的显存管理挑战。随着生成序列的延长，键值缓存的显存开销呈线性上升，与模型权重所占的静态显存相互叠加，极易引发显存容量瓶颈与碎片化问题，从而限制了模型在长序列场景下的并发推理效率。

1.3 大模型推理面临的挑战

尽管大模型在各项任务中取得了显著的效果，其高效部署与推理仍面临来自模型架构与推理服务的双重压力，两者相互交织，共同制约了系统的推理性能、部署成本与可扩展性。

模型架构：随着模型参数量增大，其模型权重大小已经超过了单张 GPU 显卡的显存容量上限，需要引入张量并行（tensor parallel, TP），将模型权重放置在多张 GPU 显卡中，甚至是不同主机节点中。而跨设备的通信与同步，带来了额外的时间开销，并且导致计算单元因等待通信同步而处于空闲状态。其次，键值缓存的大小与模型的隐藏层维度和序列长度线性相关。在长上下文场景中，键值缓存的体积可能轻易超过模型权重本身。这不仅挤占了用于计算的显存空间，限制了批处理大小，还导致了显存访问的碎片化，显著降低了系统的吞吐量。此外，预填充与解码阶段的差异性也会导致 GPU 利用率大幅波动，从而成为制约大模型推理效率的关键因素。

推理服务：大模型应用场景从单一问答向复杂任务处理转变，对底层的推理服务提出了严峻挑战。一方面，为了提高模型输出的质量，目前广泛采用检索增强生成（retrieval-augmented generation, RAG）[35] 和测试时扩展（test-time scaling, TTS）[36]，导致模型的上下文窗口及生成序列显著延长。同时，基于大模型的智能体[13] 引入了多轮交互与自主规划，大幅提升了单会话内的计算需求和模型调用次数。另一方面，在线服务需要同时处理多个用户的多个请求，这些请求的到达时间、输入长度、输出长度、服务质量要求上可能差异巨大。在这种动态负载场景，容易导致计算资源争抢和排队延迟。比如一个长预填充任

务可能会阻塞后续众多的短解码任务，导致整体服务延迟激增。若单纯按峰值流量配置算力，在流量低谷时会造成大量的资源闲置，提高了服务成本。

综上所述，为了释放大模型的生产力潜能，使其能够高效、经济、可靠地服务于广泛场景，大模型推理优化已成为学术界与工业界研究的重点方向之一。接下来本文将分别从模型层、算法层、服务层对已有的大模型推理优化技术进行分类总结，最后展望了大模型推理优化的未来发展方向。

2 模型层优化

2.1 量 化

面对大模型庞大的参数规模所带来的巨额显存占用与计算负载，模型量化成为了最直接且有效的压缩策略。模型量化是指将大模型中的浮点数转换为低精度的整数，并尽可能地减少量化产生的计算误差。比如使用 INT8 等低精度的数据类型来替代 32b 浮点数，计算单元能够一次性加载更多的数据，从而计算可以更快地完成。

根据量化应用时间的不同，量化[14]可以分为量化感知训练(quantization-aware-training, QAT)和训练后量化(post-training-quantization, PTQ)。其中 QAT 在模型训练过程中加入了量化过程，将量化损失集成到了模型训练的过程中，让模型在量化值的约束下进行模型参数更新。但在大模型场景下，QAT 本身带来的计算成本和数据依赖使其难以落地。PTQ 是在模型训练完成后进行量化，仅需利用少量校准数据来计算缩放因子、零点等量化参数，具有简单和高效的特点，是实现大模型高效推理的关键路径。因此，本文重点关注 PTQ 技术。

然而，直接应用 PTQ 面临着严峻的精度挑战。Transformer 架构中普遍存在离群值(outliers)，量化过程对其异常敏感，简单的截断或缩放极易产生不可逆的精度损失。因此，现有研究主要致力于分析 Transformer 架构下的离群值分布特性，以减少量化带来的精度下降。在大模型中，量化的对象包括了权重、激活值和键值缓存。由于权重和激活值的联系紧密，因此本文将两者放在一起讨论。

2.1.1 权重和激活值量化

如表 1 所示，大模型量化研究已从早期相对成熟的 8b 量化，逐步演进至更具挑战性的 4b 及更低比特量化，其核心矛盾始终围绕如何在压缩比特宽度的同时，最大限度地保持模型的表达能力与推理精度。本节将系统梳理该领域的关键进展，从方法论层面分析其设计思路与演进逻辑。

Table 1 Comparison of Quantization Approaches for Weights and Activations

表 1 权重和激活值的量化方法对比

量化方法	量化位数	量化对象
LLM.int8()[37]	8	权重、激活值
SmoothQuant[38]	8	权重、激活值
GPTQ[39]	3，4	权重
AWQ[40]	4	权重
QuaRot[41]	4	权重、激活值
SpinQuant[42]	4	权重、激活值
DuQuant[43]	4	权重、激活值
SpQR[44]	3，4	权重
SqueezeLLM[45]	3	权重
AQLM[46]	2，3	权重
QuIP#[47]	2，3，4	权重

大模型的激活值在特定通道上存在显著的离群值，其数值通常比正常值高出数百倍，直接使用 Min-Max 线性量化会导致模型精度大幅下降。针对这一问题，早期的研究者提出了混合精度分解方案。如 LLM.int8()[37]提出了一种向量级的量化策略，在量化过程中动态地检测离群值通道，将其分离并保持 FP16 精度进行计算，而其余部分则采用 INT8 量化。这种分而治之的策略虽然极大地保留了模型精度，但由于权重和激活值的动态拆分与计算，导致其实际推理延迟甚至高于 FP16 基线。

为了消除运行时的动态开销，SmoothQuant[38]提出了一种离线平滑策略。其核心观察在于，权重的分布较为平滑且易于量化，而激活值则表现出显著的通道依赖性，即特定的特征通道在所有输入词元上均表现出类似的离群值分布。基于这一特点，SmoothQuant 利用数学等价迁移思想，在离线阶段利用部分样本数据计算得到一个平滑因子，并通过该因子对激活值进行缩小，而权重则进行相应的放大，从而将量化难度从激活值迁移到权重上。由于权重的调整可以离线完成，在推理阶段可以直接使用高效的 INT8 矩阵乘法，相比于 FP16 基线实现了 1.56 倍的加速。

随着大模型参数规模和复杂性增加，8b 量化的压缩率已难以满足实际部署需求，4b 量化逐渐成为研究焦点。GPTQ[39]是 4b 量化的代表性工作之一，该工作建立在算法 OBQ[48]的理论基础之上：OBQ 通

过计算海森矩阵来最小化逐层量化误差，并采用贪心策略来动态调整量化顺序。然而 OBQ 需要频繁更新海森矩阵且依赖动态索引，计算复杂度高达 $O(d^4)$，其中 d 表示隐藏层维度，难以高效应用于千亿参数的大模型。GPTQ 放弃了 OBQ 中昂贵的动态排序策略，转而采用固定顺序处理权重列，逐层进行非对称量化。此外，GPTQ 引入了延迟批量更新技术，将权重矩阵的逐元素串行更新转化为分块的矩阵运算，在块内累积误差补偿量，极大地提升了 GPU 利用率，能够在单张 GPU 上若干小时内完成千亿参数模型的量化。

上述量化方案采用统一的策略处理大模型的所有权重参数，但 AWQ[40] 指出大模型的性能主要由 1% 的关键权重决定，保留这些关键权重不进行量化，其他权重采用 4b 量化方案，能够极大程度上保证模型的推理精度。然而，混合精度推理难以有效利用 Tensor Core 的 INT4 加速指令。因此 AWQ 提出了一种激活感知缩放策略，旨在实现 4b 权重的统一量化。AWQ 的关键发现在于，如果某个输入特征通道的激活值幅度很大，那么该通道对应的权重参数则属于关键权重。因此，AWQ 通过搜索最优缩放因子，缩小关键激活通道的幅度，同时放大对应的权重。这种数学等价变化使得关键权重在被量化为 INT4 时，其相对量化误差能够显著减少。

尽管 GPTQ 和 AWQ 实现了权重的 4b 量化，但激活值中的离群值使得其难以被量化至 4b，通常仍需保持 FP16 精度（W4A16）。为了实现更高效率的全链路 4b 量化（W4A4），必须解决激活值的分布问题。其中 QuaRot[41] 提出了一种基于旋转的 4b 量化范式，通过在推理过程中对权重、激活量使用计算不变的旋转变换，如随机 Hadamard 变换，将原本集中在特定通道的离群值打散，平摊到所有维度。旋转之后得到的数据分布变得更加均匀，从而降低了激活值的量化难度。然而，QuaRot 使用的旋转矩阵通常是随机生成的，未必能达到理论最优。SpinQuant[42] 进而提出通过学习的方式寻找最优旋转矩阵，相比随机旋转显著降低了量化误差，极大地缩小了量化模型与全精度模型在零样本推理任务上的差距，但同时也带来了额外的离线训练开销。DuQuant[43] 认为单一的旋转变换仍显不足，因此引入了双重变换策略，先通过旋转矩阵将离群点重新分配到相邻通道，再结合 Zigzag 排列策略均衡块间分布，通过这种旋转加排列的方式更彻底地平滑了激活值分布，进一步提升了低比特量化模型在大规模离群点存在情况下的鲁棒性。

为了进一步减少量化位数，部分研究提出了 3b 量化方案。SpQR[44] 和 SqueezeLLM[45] 都观察到大模型中存在少量敏感权重对误差贡献巨大。SpQR 对此采用了稀疏量化表示，将这些关键的离群权重分离并以高精度存储，而将其余绝大多数权重压缩至 3~4b。SqueezeLLM 则利用 Fisher 信息矩阵计算每个权重的敏感度，将权重矩阵分解为一个稀疏矩阵和一个稠密矩阵，对于包含了异常值的稀疏矩阵保留 FP16 精度，而稠密矩阵则使用非均匀量化方案。AQLM[46] 放弃了直接存储权重值，而是学习多个码本，通过多个低位向量的查表求和来高精度地逼近原始权重。QuIP#[47] 则引入了更复杂的非相干处理理论，首先利用随机正交变换把原本分布不均的权重转换成均匀的球形分布，然后结合 E8 格这一特殊的几何结构将向量打包。

总体而言，目前 4b 量化凭借在显存占用与精度保持间的优异平衡，已取代 8b 量化成为当前大模型高效部署的主流选择。而 3b 及以下的极低精度方案则展现了巨大的潜力，仍处于活跃探索阶段。

2.1.2 键值缓存量化

为了提升推理效率，主流推理系统普遍对每个词元产生的键（key）和值（value）向量进行复用，即键值缓存。然而，随着模型参数规模的不断扩大以及推理序列的不断增长，键值缓存的规模会急剧上升。例如 DeepSeek-R1 模型在 32K 序列长度下，键值缓存占用了约 53 GB 显存。键值缓存量化旨在降低推理过程中生成的键和值的数值精度，从而减少显存占用并提升推理吞吐量。键值缓存虽然作用于推理阶段动态生成的中间状态，但其量化本质是对张量数值精度的压缩，因此本文将其与权重、激活值量化并列讨论。

如图 4 所示，学术界针对键值缓存的数据特点提出了一系列量化策略。KIVI[49] 分析了键值缓存的分布特征，发现键缓存在特定通道上表现出显著的离群值特征，而值缓存的分布相对平缓。KIVI 针对上述发现提出了一种无需微调的非对称 2b 量化方案，即对键缓存采用逐通道量化，对值缓存则采用逐词元量化，这样可以将键缓存的离群值限定在当前通道内，而不对其他通道产生影响。为了进一步保持模型的推理性能，KIVI 保留了全精度的滑动窗口，让最近的上下文保持全精度推理。

KVQuant[50] 进一步研究了键缓存量化的难点，指出目前使用的旋转位置编码会导致不同通道信息的混合，从而导致量化困难。对此 KVQuant 提出了 Pre-

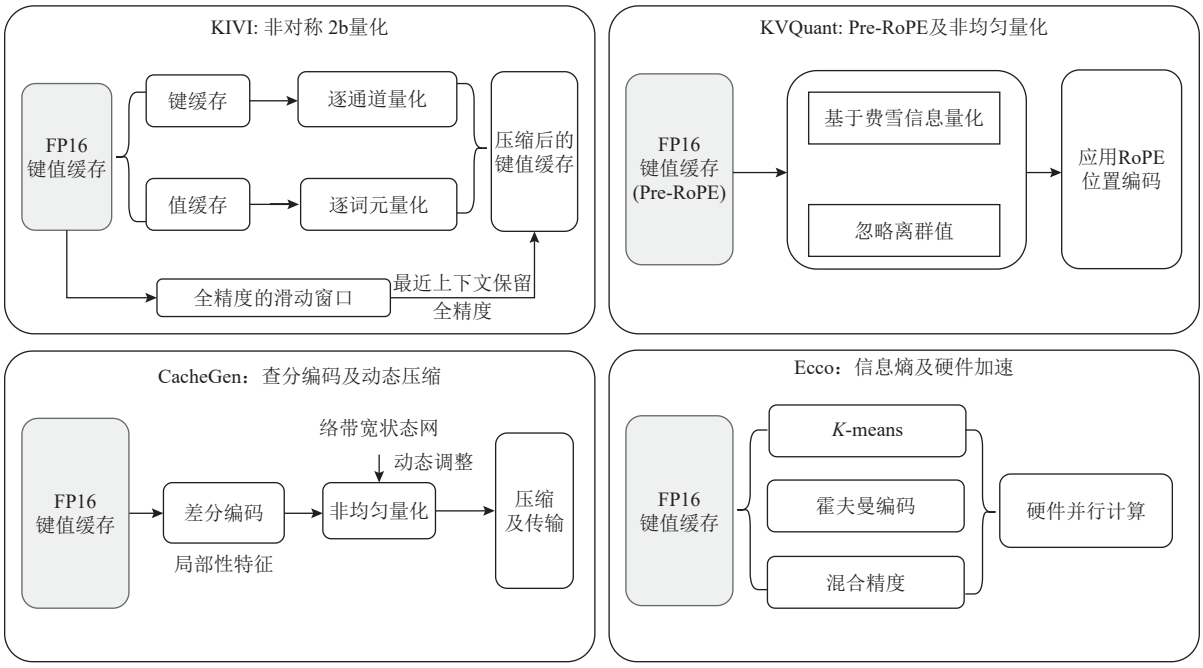


Fig. 4 Comparison of prevalent weight and activation quantization methods

图 4 主流权重与激活值量化方法对比

RoPE 的量化策略，在应用旋转位置编码之前对键缓存进行量化。除此之外，KVQuant 认为键值缓存是非均匀分布的，因此结合了基于费雪信息的非均匀量化方法对键值缓存进行量化，并在量化过程中截断离群值，实现了在 2b 精度下的高效推理。

除了关注显存容量的量化方案，部分研究也通过编码技术以降低键值缓存传输延迟和显存带宽瓶颈。CacheGen[51] 针对上下文复用场景下的网络传输延迟，将键值缓存视作比特流进行压缩。CacheGen 利用词元间的局部性特征进行差分编码，并结合了非均匀量化策略，根据网络带宽动态调整压缩等级，显著降低了长上下文的加载时间。Ecco[52] 则从信息熵的角度出发，认为现有的均匀量化没有充分利用模型权重的分布特征，从而提出了一种基于霍夫曼码本的非均匀量化策略。Ecco 首先利用 *K*-means 聚类提取出全局共享的权重分布模式，针对不同层的注意力头的特征，对键值向量采用混合精度量化，在压缩率和精度之间取得平衡。且 Ecco 设计了高效的并行解压算子，利用多级流水线和重叠解码技术，实现了极高的吞吐量。

2.2 模型稀疏化

大模型的强大性能往往建立在巨大的参数规模之上。但研究表明，目前大模型普遍存在严重的过参数化现象。在推理过程中，并非所有的参数都参与了有效的特征提取与变换，大量的权重参数对于输出的贡献微乎其微。这种模型内部的冗余性为提升大模型推理效率提供了巨大机会。基于这一洞察，模型稀疏化技术通过剔除冗余参数，从而将稠密矩阵转化为稀疏矩阵，能够在降低计算开销与显存占用的条件下，达到原模型相近的性能。

从作用机制以及模型形态来看，稀疏化技术主要包括了静态稀疏化以及动态稀疏化。其中静态稀疏化也称模型剪枝(pruning)[53]，即在推理前通过启发式的指标来移除权重矩阵中的冗余元素。随着 SparseGPT[54] 和 Wanda[55] 等算法的提出，后训练稀疏化逐渐成为主流，允许在无需重新训练的情况下直接获取稀疏模型。动态稀疏化则是以混合专家模型(mixture-of-expert, MoE)[56] 为代表的原生稀疏架构。不同于剪枝的永久删除，原生稀疏架构通过动态路由机制，仅激活部分参数参与计算，能够在一定计算预算的条件下，扩大模型参数规模以提升其性能。

2.2.1 静态稀疏化

如图 5 所示，模型剪枝可以分为结构化剪枝和非结构化剪枝。SparseGPT[54] 是首个针对大规模 GPT 模型设计的非结构化剪枝算法，将剪枝问题转化为大规模的稀疏回归问题。在处理过程中，SparseGPT 利用了与 GPTQ 类似的数学变换，即利用海森矩阵来评估权重的重要性，并通过其逆矩阵来更新剪枝后的权重，以补偿剪枝后带来的误差。此外，SparseGPT 通过高斯消元法在不同权重之间复用海森矩阵的逆，

有效降低了计算复杂度，但其实现复杂度和内存开销仍然存在一定门槛。Wanda[55] 观察到如果一个权重对应的输入激活值很大，即使该权重本身的幅度很小，其对输出仍然会产生显著影响。因此 Wanda 绕过了海森矩阵，通过计算权重绝对值与对应输入激活值 L2 范数来评估重要性，以更低的计算成本达到了与 SparseGPT 相当的性能。

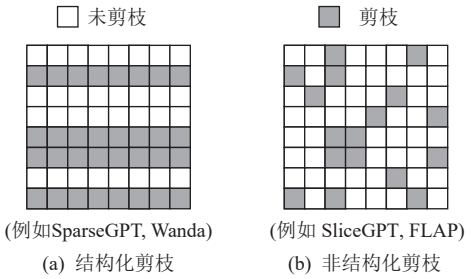


Fig. 5 Contrast between unstructured and structured pruning

图 5 非结构化剪枝与结构化剪枝对比

上述方法关注于非结构化剪枝，虽然减少了模型的显存占用，但由于其稀疏分布的不规则性，在通用硬件上难以直接转化为推理加速。SliceGPT[57] 提出了一种结构化剪枝方法，旨在通过移除整个行或列来减少模型的嵌入维度，从而生成一个更小的密集型模型，能够在通用硬件上实现加速。然而直接删除行或列会对模型造成巨大破坏，导致输出产生极大误差。SliceGPT 利用了 Transformer 架构中的计算不变性，即在 RMSNorm 操作前后插入正交变换矩阵，模型的输出保持不变。基于该特性，SliceGPT 计算每层输入的主要成分，将网络投影到主要成分上，剪去奇异值较小的行和列，从而在保持模型性能的同时实现了高效的结构化压缩。

FLAP[58] 同样致力于解决大模型结构化剪枝问题。与 SliceGPT 不同，FLAP 发现某些通道的特征在不同样本间表现出极低的波动，意味着它们对输出的贡献相对恒定，可以通过一个静态的偏置项来代替权重参数。因此 FLAP 提出了一种基于波动的剪枝方法，优先剪去那些波动性小的列，并利用偏置项来补偿剪枝带来的误差。为了解决不同层之间重要性难以比较的问题，FLAP 对度量分数进行了标准化处理，从而实现了全局自适应的结构化剪枝。

2.2.2 动态稀疏化

根据缩放定律（scaling law）[2]，模型质量与参数量呈现出幂律关系。但参数量增长在提升性能的同时，也不可避免地导致了计算成本的增加，使得推理延迟和资源消耗随之增长。为了平衡模型规模与计算效率，原生稀疏架构通过门控网络引入了动态稀疏机制，对于特定的输入选择性激活一部分参数进行计算。如图 6 所示，根据稀疏化作用的位置不同，已有的原生稀疏架构可以分为 MoE（mixture of experts），MoA（mixture of attention），MoD（mixture of depths）。其中 MoE 将前馈网络层替换为多个并行专家层，MoA 动态选择注意力机制中的键值对进行计算，MoD 则是动态决定是否跳过某些计算层。

谷歌公司提出的 GShard[56] 是 MoE 发展史上的一个里程碑，其将每隔一个 Transformer 层的前馈网络替换为 MoE 层，每个词元被路由到 2 个专家。为了防止专家坍缩与负载不均，GShard 设计了负载均衡损失函数，对负载不均的情况做进一步惩罚。

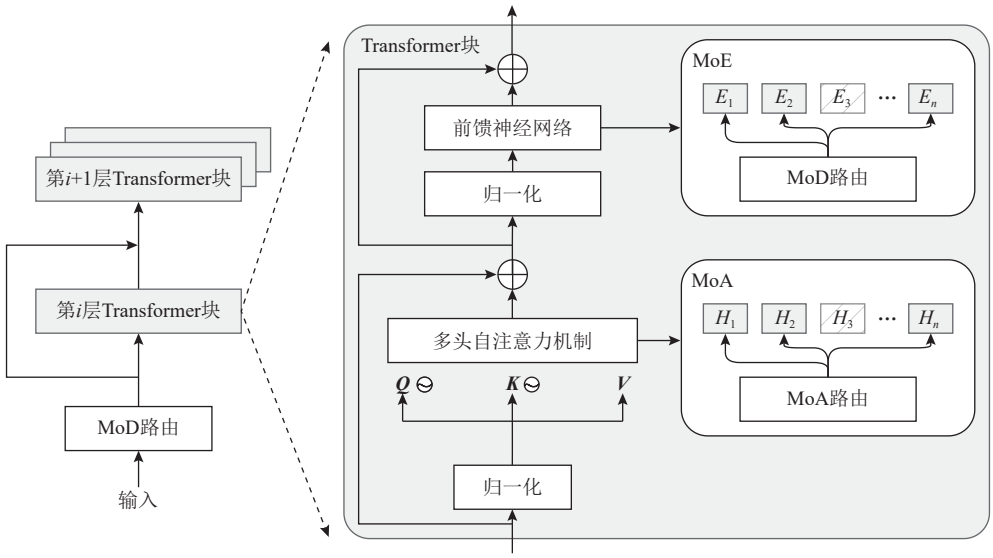


Fig. 6 Native sparse architectures: MoE, MoA, and MoD

图 6 原生稀疏架构 MoE，MoA，MoD

GShard 同时引入了专家容量的概念,保证每个专家分配到的词元不超过该容量上限,多余的词元会被强制丢弃或者直接通过残差连接传递到下一层。GLaM[59] 在 GShard 的基础上,通过极高比例的稀疏化设计进一步释放了 MoE 的潜力。在 1.2 万亿的总参数规模下,其门控机制使每个词元仅激活约 8% 的参数即可完成计算,并且在少样本学习任务中实现比肩 GPT-3 的性能。Switch Transformer[60] 认为路由到多个专家会增加计算和通信负担,并且精度提升有限。因此采用了极简的 Top-1 路由策略,即每个词元仅被路由至一个概率最高的专家。

然而,传统的辅助负载均衡损失往往会干扰模型的主要训练目标,导致性能受损。针对这一问题,DeepSeek-V3[61] 创新性地提出了无辅助损失负载均衡策略。不同于 GShard 强行在训练过程中加入惩罚项,DeepSeek 通过在路由得分上增加一个动态可学习的偏置项来调节专家负载。这种机制在确保专家负载均衡的同时,避免了辅助损失对梯度下降的干扰,从而显著提升了模型的收敛效果与最终性能。此外,Mixtral[62] 发现专家负载展示出一种基于句法和局部性的分布模式,比如连续的词元在同一层中倾向于路由到同一个专家。

上述研究主要致力于优化专家路由策略,而专家模块本身的架构设计,即专家的粒度与功能划分,同样是决定 MoE 模型性能上限的关键因素。其中 GShard, GLaM, Switch Transformer 的专家数量通常较少,为 8~16 个,每个专家的参数量极大。而 DeepSeek 模型系列进一步将专家拆解的更细,比如 DeepSeek-V3 中拥有 256 个专家。为了解决稀疏专家可能遗漏通用知识的问题,DeepSeek 引入了共享专家机制,这些专家在所有词元的计算中总是被激活,负责捕获通用的句法和语法知识,而其余专家则负责捕获专用知识。除了基于词元特性来选择专家,部分工作也探索了让专家来选择词元,即每个专家选择其最擅长的 Top-k 个词元。

不同于 MoE 通过稀疏化前馈网络层来实现计算效率的提升,MoA 聚焦于注意力层的稀疏化。Moonshot AI 提出的 MoBA(mixture of block attention)[63] 将历史上下文划分为固定大小的块,门控网络负责计算输入与不同上下文块之间的亲和度,仅选择 Top-k 个最相关的上下文块中的键值进行注意力计算。在推理过程中,MoBA 需要保留最近的上下文块以确保局部信息的完整,从而维持模型性能。相比于 MoBA 中词元选择历史上下文块,MoSA(mixture of sparse attention)[64] 采用了注意力头选择路由机制。MoSA 将每个注意力头视为一个独立的专家,让注意力头自主地从输入序列中选择最相关的 Top-k 个词元进行处理,而不是由词元来选择注意力头。这种反向选择的机制允许模型在相同的计算成本下增加注意力头的数量,从而提升模型的性能。除此之外,DeepSeek 提出了一种分层稀疏注意力架构 NSA(native sparse attention)[65]。NSA 通过压缩历史上下文捕捉细粒度的全局语义,通过分块交互捕捉细粒度的关键依赖,通过滑动窗口捕捉局部上下文。这种多粒度的上下文建模能够更好地平衡计算效率与长距离信息捕捉能力。

谷歌提出的 MoD(mixture of depth)[66] 认为并非所有词元都需要所有 Transformer 层参与计算,因而提出了基于层级的动态路由机制。该路由机制为每个 Transformer 层设定了固定的容量,动态地识别出最重要的 Top-k 个词元参与该层的计算,剩余词元则直接通过残差连接传递到下一层。这种设计确保了计算图和张量大小的静态性,从而能够提高硬件效率。

2.3 高效架构演进

尽管前文所述的量化与稀疏化方法能够显著降低显存占用与计算开销,但 Transformer 架构核心的自注意力机制仍面临着固有的效率瓶颈。一方面,随着序列长度的增加,标准多头注意力带来的键值缓存显存占用呈线性增长,极易形成显存墙;另一方面,其计算复杂度随序列长度呈 2 次方增长,限制了模型在超长上下文场景下的推理性能。为了从根本上突破这一架构限制,研究者们开始探索更底层的架构演进方案。本节将从 2 个维度探讨高效架构的设计:一是在保持 2 次注意力框架的前提下,通过改变键值的映射关系来降低访存压力(如 MQA, GQA, MLA)[16-18];二是彻底重构序列建模方式,采用线性复杂度或混合架构实现对超长上下文的高效处理。

2.3.1 注意力访存优化

Transformer 中默认的 MHA 为每个注意力头都分配了独立的查询、键和值投影矩阵。在数学上,MHA 等价于多个独立的单头注意力的拼接。这种设计虽然赋予了模型捕捉多样化特征的强大能力,但是在长序列推理中,每个解码步骤都需要读取全量的键值缓存。从硬件效率来看,MHA 读取了大量的内容,但是计算次数却很少,其计算强度较低,属于典型的显存受限场景。MQA, GQA, MLA 的核心目的在于减少键值缓存的读取,同时尽可能地保持模型效果。

如图 7 所示，MQA[16] 采用了激进的压缩策略，所有的查询头共享同一组键值对。MQA 将键值缓存减少到了原来的1/h，其中h为注意力头数量。假定 MHA 的计算强度为 1，则 MQA 的计算强度提高了h倍，能够充分利用 GPU 的计算能力以提升推理速度。但 1 对多的共享机制也削弱了模型的上下文特征捕捉能力，导致模型性能出现了下降。GQA[17] 则作为 MHA 和 MQA 之间的过渡方案被提出，其主要思想是将所有注意力头划分为不同组，每组共享同一对键值缓存。当分组数设置合理时，GQA 能够在保持接近 MHA 的特征捕捉能力的同时，显著降低显存带宽需求并提升推理速度。注意，GQA 分组个数是 1 个超参数，组数越大，其模型精度越接近 MHA，但推理延迟也相应增加，反之则趋向于 MQA。

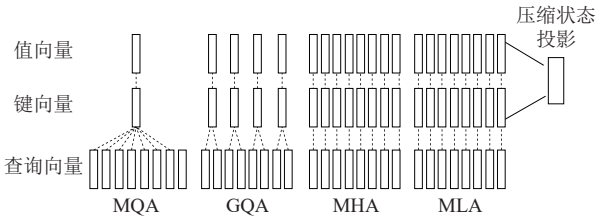


Fig. 7 The evolution of attention architectures[18]

图 7 注意力架构的演进[18]

不同于 GQA 仅在注意力头上做减法，MLA[18] 引入了低秩键值联合压缩技术，将键值缓存投影到低维向量空间。这种设计本质上是将原本受限于带宽的访存开销转化为计算开销，模型只需从显存加载压缩后的低维向量，然后在芯片内部通过矩阵乘法将其还原。这种转换大幅提升了算子的计算强度，在大幅降低显存占用的同时，充分释放了 GPU 并行计算能力，并通过多头机制保证了优于 GQA 的模型性能。但由于 MLA 使用了低秩压缩，其性能相比于 MHA 仍然存在一定的损失。

综上，MQA, GQA, MLA 的演进，其核心脉络在于持续提升解码阶段注意力算子的计算强度。未来的注意力架构将进一步压缩键值缓存的访存需求，使解码阶段从访存受限逐步转向计算受限，从而更充分地释放现代 GPU 的并行算力。

2.3.2 线性复杂度架构

上述不同注意力架构的主要目的就是降低键值缓存的大小以提升计算效率，但注意力机制仍受限于随序列长度增长的 2 次方计算复杂度，难以处理超长上下文场景。因此学术界开始研究线性复杂度的架构，以更好地支持目前超长上下文场景，代表方法包括 RWKV[19] 和 Mamba[67] 系列。其中 RWKV 将 Transformer 的并行训练优势与 RNN 的高效推理相结合，提出了一种线性架构的 WKV 算子。WKV 算子在训练阶段可以实现类似于 Transformer 的并行计算，在推理阶段则展开为 RNN 模式，将历史信息压缩至固定大小的隐状态中，仅需常数级的显存占用即可处理无限长的序列内容，极大地提升了计算强度。

Mamba 则引入了结构化状态空间模型理论（state space model, SSM）。传统的 SSM 模型处理每个词元的方式是恒定的，即每个词元的信息保留或遗忘的速度恒定，不随输入内容改变，因此缺乏基于内容的上下文推理能力。Mamba 通过选择机制，将传统 SSM 模型中的关键参数从定值转化为依赖输入词元的函数，从而使模型能够根据当前词元有选择地传播或遗忘信息。Mamba-2[68] 进一步突出了结构化状态空间对偶性理论，证明了 SSM 与某种形式的掩码注意力在数学上是等价的。Mamba-2 利用这种对偶性，通过简化状态矩阵的结构，将原本依赖串行扫描的递归计算转化为分块的矩阵乘法操作，从而能够充分利用 GPU 的计算单元。

上述 RWKV 和 Mamba 系列可以视为以线性复杂度为目标的序列建模架构，但完全放弃标准注意力机制会导致局部细粒度信息捕捉能力的下降。为此，Munkhdalai 等人[69] 提出了 Infini-attention，在单个 Transformer Block 中融合局部注意力与长期线性注意力 2 类互补建模能力。局部注意力负责建模当前分段内的词元间依赖关系，历史分段产生的键值信息将被压缩到固定大小的记忆矩阵中。长期线性注意力基于记忆矩阵检索历史上下文信息，并通过可学习的门控机制与局部注意力的输出融合。这种设计使得模型的记忆占用与序列长度无关，能够即插即用地集成到现有 Transformer 模型中，无需从头训练。

此外，部分前沿模型开始探索层级混合架构，将大部分 Transformer 层中的标准注意力替换为线性注意力，以权衡模型精度和计算效率。如表 2 所示，MiniMax M1[70], Qwen3-Next[71], Kimi Linear[72] 是这一范式的 3 个代表性工作，它们分别在线性注意力算子、标准注意力算子以及混合比例上进行了不同探

Table 2 Comparison of Hybrid Linear Attention Models

表 2 混合线性注意力模型对比

模型	线性注意力算子	标准注意力算子	混合比例
MiniMax M1	Lightning Attention	GQA	7：1
Qwen3-Next	Gated DeltaNet	Gated Attention	3：1
Kimi Linear	Kimi Delta Attention	MLA	3：1

索。线性注意力的本质就是将键值缓存压缩为一个固定大小的状态矩阵，每步根据新生成词元的键值来更新它，不同算子的主要差异体现在状态更新规则上。

Lighting Attention 主要采用累积式状态更新方式，将历史词元的信息持续写入固定大小的状态矩阵，并通过 I/O 感知的实现方式优化状态矩阵的计算过程，从而提升长序列场景下的执行效率。相比之下，Gated DeltaNet 每次仅更新状态矩阵中与当前词元相关的部分，并使用标准门控机制实现自适应遗忘机制。Kimi Delta Attention 进一步将门控机制细化到特征通道维度，为每个特征通道分配独立的遗忘速率，使无关特征能够被快速衰减，而关键特征则可在状态矩阵中长期保留。由此可见，混合线性注意力架构的发展趋势正在从简单的状态累积，转向更加细粒度、内容自适应的状态选择与更新机制。

除了通过线性注意力降低长序列建模复杂度，另一类工作从分布式系统角度扩展标准注意力的可处理长度。Ring Attention[73] 将输入序列切分到多个设备上，各设备按环形拓扑传递键值块，并将通信与分块注意力计算重叠执行，从而在不引入注意力近似的前提下，使上下文长度随设备数量近似线性扩展。RingX[74] 进一步针对 Ring Attention 在大规模 HPC 场景下面临的点对点通信开销和负载不均问题，优化了序列划分、集合通信和负载均衡策略，提升了超长上下文训练中的通信效率与设备利用率。

3 算法层优化

尽管模型层的量化与稀疏化方法有效降低了静态参数的资源需求，但大模型推理的执行过程仍面临着计算逻辑低效与硬件算力浪费的矛盾。这种矛盾源于 Transformer 架构对长序列数据的显存依赖以及自回归生成机制的串行束缚。如图 8 所示，本文以大模型推理的数据流转为主线，从上下文压缩、解码加速与算子优化 3 个维度，对算法层优化方法进行了系统综述。首先，针对输入侧与运行时积累的海量上下文数据，上下文压缩方法旨在通过剔除无用信息缓解显存容量压力（图示①）；其次，针对输出侧自回归生成带来的高延迟，解码加速策略通过打破严格的串行依赖实现生成吞吐量的提升（图示②）；最后，算子优化则致力于通过重构底层计算范式突破显存带宽瓶颈，实现硬件利用率的极致优化（图示③）。

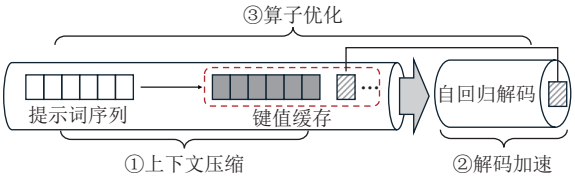


Fig. 8 Framework for algorithmic-level optimization in LLMs inference

图 8 大模型推理算法层优化框架

3.1 上下文压缩

在大模型的实际应用中，输入到模型中的提示词序列至关重要，大量研究提出了设计高效提示词的方案，并通过实验表明精心设计的提示可以释放大模型的能力。比如上下文学习（in-context learning, ICL）[75] 在提示词中包含多个相关的示例，RAG[35] 从知识库中检索到问题相关的内容以丰富上下文，智能体应用中通常要输入已有的对话记录与复杂的工具定义进行下一步决策。上述技术的目的是提升模型的回答质量，但同时也带来了超长上下文的推理场景，推理开销极大，模型甚至可能遗忘了部分关键内容，从而导致准确性下降。Anthropic 的研究数据表明，当工具定义超过 50 个时，词元消耗会呈指数级上升，但准确性却直线下滑[76]。针对上述问题，部分研究提出了上下文压缩，在维持模型性能的同时，降低推理开销。从压缩机制的作用时机角度，上下文压缩可分为静态压缩与动态压缩 2 大范式。如图 9 和图 10 所示，静态压缩是指在提示词输入到大模型之前，将提示词压缩为一个精简版本，动态压缩则是在解码阶段动态地略过或者驱逐部分键值缓存以降低计算开销。

3.1.1 静态压缩

信息量是信息论中的一个基本概念，它量化了一个事件所传递的信息量。在语言建模领域，信息量被定义为生成下一个词元时概率的负对数，也被称为词元的困惑度。Selective Context[78] 认为低困惑度的内容是可以被推断出来的冗余信息，因而通过一个小模型来计算提示词中每个词元的困惑度，通过设定阈值移除低困惑度的部分。通过该压缩方式，Selective Context 能够在保留核心语义的同时显著减少显存占用和推理延迟。

LLMLingua[79] 认为 Selective Context 忽略了提示词的不同部分对压缩的敏感性不同，以及使用的小模型与目标大模型之间存在分布差异。为此 LLMLingua 提出了一个变粒度的压缩框架来解决这些问题。LLMLingua 首先设计了一个预算控制器，

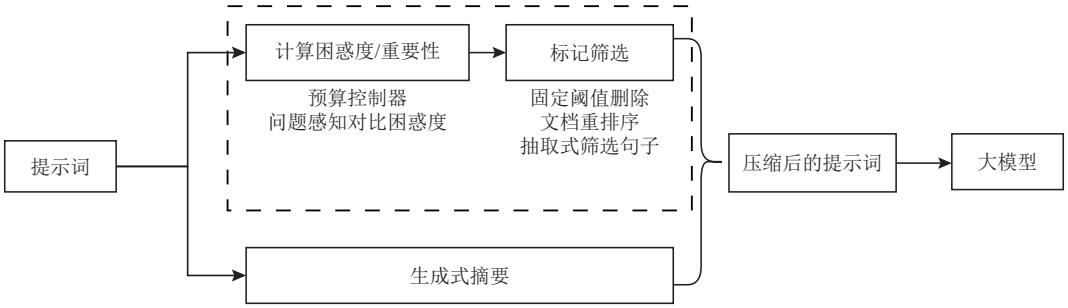


Fig. 9 Workflow of static prompt compression

图 9 静态提示词压缩工作流程

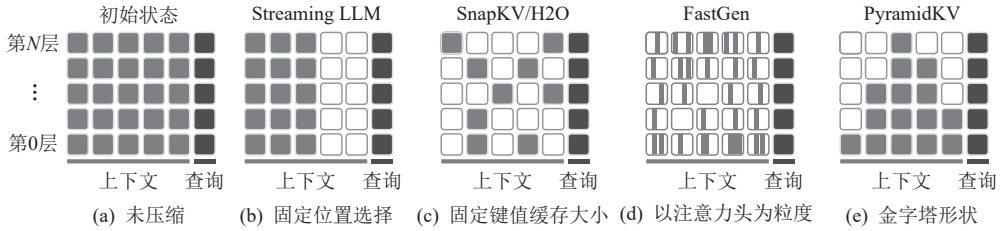


Fig. 10 Comparison of different dynamic KV cache eviction policies[77]

图 10 不同动态键值缓存驱逐策略对比[77]

为提示词中的不同组成部分分配不同的压缩率，比如给予系统指令和问题指令更低的压缩率。随后引入了词元级别的迭代压缩算法，将已经压缩过的上下文输入到小模型中，有效地保留了词元之间的条件依赖关系。为了缓解小模型和大模型之间的分布差异，LLMLingua 通过微调小模型进行对齐，进一步提高了生成质量。

针对 RAG 等长上下文场景，LongLLMLingua[80] 通过提升提示词的信息密度，从而提升大模型的性能。具体实现上，LongLLMLingua 设计了问题感知的词元重要性评估算法，计算给定问题条件下的困惑度与无条件困惑度的差值，以此评估词元对特定问题的重要性。同时采用了文档重排序策略，将高相关性的内容放在提示词的开头和结尾，以缓解大模型的位置偏差。当大模型在回答中需要引用人名、地名等原始实体时，由于提示词已被压缩，可能导致输出不完整。为此 LongLLMLingua 使用了最大公共最短子序列将模型输出与原始输入进行匹配，从而恢复被压缩的实体。

相较于上述方法删除提示词中的部分词元，RECOMP[81] 则提出将提示词进行摘要，生成简明的文本，再输入到大模型中。摘要的方式通常分为抽取式和生成式，因此 RECOMP 也采用了这 2 种实现形式：一是通过对比学习训练的抽取式压缩器，筛选出对问题最有帮助的句子；二是使用知识蒸馏训练出一个生成式压缩模型，如果文档与问题不相关则输出空字符串，否则输出文档的摘要。

3.1.2 动态压缩

尽管静态压缩能够有效缩短初始输入的长度，但在长序列生成过程中，不断累积的键值缓存依然会造成巨大的显存压力。为此，上下文的动态压缩旨在推理的解码阶段动态地识别并保留关键信息，剔除无关的历史状态，从而在保证模型长上下文生成能力的同时，显著降低显存占用与计算开销。基于这一核心思想，现有工作在不同部署场景下发展出 2 种操作模式：1）在纯 GPU 推理场景下，直接将不重要的历史词元从缓存中丢弃以释放显存；2）在键值缓存卸载场景下，则保留完整缓存，仅选择性地将重要词元加载回 GPU 以降低 I/O 开销。

在纯 GPU 推理场景下，动态压缩主要表现为键值缓存驱逐，即依据注意力分布、词元重要性或模型层级特征，从有限显存中选择性保留关键历史词元。StreamingLLM[82] 首次揭示了 Attention Sink 现象，即序列起始位置的词元在任何条件下均维持较高的注意力权重，同时注意力分布表现出对邻近词元的局部偏好，也就是注意力矩阵对角线位置的值相对更大。因此 StreamingLLM 在解码过程中仅保留最新的滑动窗口中的内容以及序列起始位置的若干个词元，从而在固定计算资源的条件下实现了无限长度推理。但直接丢弃中间序列必然会导致关键语义信息的丢失。在此背景下，H2O[83] 发现词元的重要性服从幂律分布，少数词元接收到了绝大部分注意力值，进而

提出了一种贪婪的驱逐策略。其在解码过程中动态保留注意力分数最高的词元以及最新的内容，从而在大幅压缩键值缓存的同时有效地维持了模型性能。

FastGen[84]，SnapKV[77]，PyramidKV[85]则通过利用注意力机制的结构化特征来优化压缩效率。FastGen发现不同的注意力头具有不同的行为。因此在预填充阶段结束时，以注意力头为粒度对模型的注意力进行分析，自适应地为不同的注意力头分配不同的压缩策略。SnapKV发现模型对提示词的关注点在预填充阶段结束之后就已经确定，解码阶段保持与预填充阶段类似的注意力分布模式，且这种模式可以通过提示词末尾的一个小窗口获取，该窗口称为观察窗口。根据该特性，SnapKV在生成前通过观察窗口计算每个词元的注意力得分均值，作为重要性的评判依据。由于观察窗口的计算内置于预填充阶段，引入的额外开销极小，显著提升了推理效率。PyramidKV则深入到了Transformer的垂直维度，指出大模型底层的注意力分布分散且宽泛，负责聚合全局信息，因此需要保留更多缓存，而高层的注意力则高度集中在少数关键词元上。因此，PyramidKV采用金字塔状的缓存分配策略，在底层保留更多键值缓存，在顶层大幅削减，从而在12%的压缩率下依然保持了优于H2O和SnapKV的性能。

上述方法均聚焦于在GPU内部直接驱逐不重要的历史词元。然而，当模型面对超长上下文或高并发前缀复用场景时，键值缓存需要进一步卸载至CPU内存甚至SSD。此时，显存容量不再是唯一瓶颈，跨层级I/O带宽与数据加载延迟反而成为制约推理效率的关键因素。在此场景下，重要性感知的动态压缩思想从丢弃不重要词元，进一步扩展为选择性加载重要词元。完整键值缓存保留在低速存储中，而推理时仅将当前计算所需的重要键值加载至GPU，从而在减少永久性信息丢失的同时降低I/O开销。

InfiniGen[86]针对键值缓存卸载至CPU内存时PCIe带宽成为瓶颈这一问题，提出了推测性预取机制。其核心观察是，下一层的重要词元可以在当前层通过少量计算提前预测出来。因此InfiniGen利用当前层的输入与下一层权重的部分近似，推测性地识别出下一层注意力中最重要的词元索引，并将这些词元对应的键值从CPU预取回GPU，而非全量加载。该预取机制显著减少了键值缓存传输量，在尽量保持模型精度的同时，实现了最高3倍的推理加速。

在生产级部署场景下，键值缓存的规模往往超出CPU内存容量，必须进一步下沉至SSD，而磁盘I/O带宽显著低于内存带宽，即使启用前缀复用机制也难以降低TTFT。IMPRESS[87]为此设计了基于重要性感知的多级前缀键值存储方案。IMPRESS的动机在于，同一注意力层内不同头的重要词元索引集合高度相似，因此可以仅加载少量注意力头的键来识别重要词元，并近似推断其他注意力头中的重要键值，从而降低重要性识别本身的I/O开销。同时，IMPRESS将重要的键值按位置重新打包为连续块，以减小读放大，并设计了结合访问频率与重要词元占比的评分机制，用于3级存储之间的数据换入与换出，有效缓解了SSD级卸载带来的I/O瓶颈。

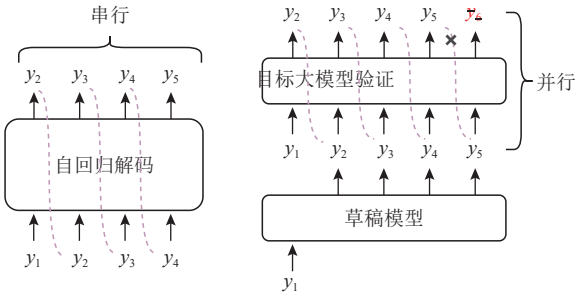
3.2 解码加速

上下文压缩有效缓解了长序列输入带来的计算压力与显存占用，但大模型在解码阶段的效率瓶颈依然显著。由于大模型目前的自回归特性，模型必须严格依赖前序序列逐个生成下一个词元。这种串行的依赖关系导致解码阶段受限于显存带宽。为了提高解码阶段的效率，研究者提出了多种解码加速策略，本文主要聚焦于投机解码和推理时计算扩展优化2种主要方法。

3.2.1 投机解码

如图11所示，投机解码使用了2个模型：一个是目标大模型(target model)；另一个参数量远小于目标大模型的草稿模型(draft model)。在解码过程中，草稿模型根据已有序列生成若干个候选词元，然后大模型通过1次前向传播对所有候选词元进行验证，定位其与草稿模型的首个分歧位置，并接受该位置之前的全部词元，其余词元则丢弃，开始下一步的草稿模型生成。投机解码能够在保持与大模型输出分布不变的前提下，有效降低推理延迟，其关键在于候选序列的生成与验证过程。

从草稿生成方法来看，投机解码经历了从独立小模型生成到大模型自生成再到基于检索方法的演



(a) 自回归解码 (b) 投机解码

Fig. 11 Comparison of autoregressive decoding and speculative decoding[88]

图11 自回归解码和投机解码对比[88]

进。Speculative Decoding[88]依赖一个独立的草稿模型来生成候选序列。为了使草稿模型与大模型的分布尽可能一致，其草稿模型的选择是大模型的低参数版本。而 Eagle 系列[89]则使用一个轻量级的解码器来生成草稿。具体而言，Eagle 将词元序列在大模型中的顶层特征作为输入，通过轻量级网络预测下一个位置的特征向量，最后复用大模型原有的输出层将特征映射为词元，显著提升了候选序列的准确率与接受率。SpecInfer[90]则进一步探索了候选序列的多样性，利用扩展和合并策略，既可以利用单一小模型的 Top-k 扩展构建多个候选序列，也可以集成多个小模型的预测结果，从而最大化候选序列的覆盖范围。

然而，引入独立小模型增加了系统维护的复杂性，且受限于小模型的生成能力，草稿序列的接受率仍然偏低。针对上述问题，Medusa[91]不再依赖独立小模型，而是在原始大模型的顶层集成多个解码头。通过单次前向传播，Medusa 能够并行预测后续多个词元。为了最大化计算效率，Medusa 提取了各解码头的 Top-k 个预测结果，并通过笛卡儿积生成多条候选序列。

不同于上述基于模型的方法，REST[92]采用了一种基于检索的范式。REST 假设当前的输入上下文常与历史数据存在重叠，比如多轮对话场景，因而直接通过字符串匹配算法从历史数据中检索匹配的最长后缀序列作为候选序列，实现了即插即用的推理加速。

在草稿验证阶段，传统的链式验证顺序处理候选词元。一旦首个词元被拒绝，后续的计算即被浪费。为了充分利用草稿的多样性，SpecInfer 引入了基于树的并行解码机制（tree-based parallel decoding），将多条候选序列组织为 1 个令牌树，通过拓扑感知掩码策略，在 1 次大模型前向传播中同时验证整个令牌树，显著提升了验证效率。Medusa，REST 等方法同样采用了类似的树状注意力机制来并行处理多条候选序列。在候选词元的接受策略上，除了经典的拒绝采样以保证分布一致性外，Medusa 认为若候选词元的概率高于一定阈值或当前上下文的熵值较高，那么应该放宽接受标准，允许更多的词元被接受。

3.2.2 推理时计算扩展优化

推理时计算扩展（test-time scaling）是指通过增加计算资源，达到更高推理质量，其典型技术路径包括 Best-of-N 采样、多数投票以及思维链（chain-of-thought）[93]等。其中，思维链通过生成中间推理思考过程来推导出最终答案，能够极大激发模型本身的推理潜能，是当前大模型中应用最为广泛的推理时扩展方式，也是本节讨论的重点。

随着思维链长度增加，模型性能并非持续提升，反而出现过度思考（overthinking）现象，冗余的推理步骤不仅显著增加计算成本与延迟，还可能引导模型偏离正确路径，最终损害准确性[94]。因此，如何在保证推理质量的前提下压缩思维链长度、或根据任务难度自适应地分配推理算力，已成为大推理模型高效部署的核心问题。围绕这一目标，现有工作沿 3 条技术路径展开：一是推理时预算控制，即在不改变模型参数的情况下，通过早停或强制延续等方式显式调节思维链长度；二是训练重构，通过监督微调或强化学习让模型内化更精简的思考模式；三是自适应推理，让模型根据输入难度动态切换思考深度。

在推理时预算控制方向上，DEER[95]指出长思维链中仅需部分推理即可得出正确结论，因此利用模型在推理转折时产生的特殊词元（如 wait, alternatively）或熵值变化作为转换点。在这些转换点引导模型直接输出中间答案，并评估其置信度。一旦置信度超过阈值，模型终止思考并输出答案。与 DEER 基于模型内部信号触发退出不同，Budget Forcing[96]从外部显式控制思维链长度。该方法利用推理模型通常通过特殊终止词结束思考的特点，在模型过早停止思考时删除终止词元，迫使模型继续推理，而当模型思考过长时，则强制截断并输出答案。由此，Budget Forcing 将原本由模型隐式决定的思维链长度转化为可控的推理预算，使同一模型能够根据不同服务场景在推理质量与响应延迟之间进行灵活权衡。

除推理时干预外，另一类方法通过训练，让模型在参数中内化更高效的推理模式。其中 MinD[23]认为思维链中包含了多个思维单元，因而将思维链过程重构为结构化的多轮交互形式。具体而言，MinD 通过有监督微调，将推理过程分解为思考单元与中间答案的多轮次结构，使得每一轮思考都输出一个阶段性结论。在此基础上，MinD 引入了强化学习方法，激励模型在保证答案正确的前提下使用尽可能少的推理轮次。ThinkPrune[97]指出简单的截断或提前退出可能会导致性能下降，因而提出了基于强化学习的思维链压缩手段，在训练中对超过规定思考长度的输出给予惩罚，迫使模型学习如何以更精简的步骤解决问题。此外，ThinkPrune 采用了迭代式训练策略，逐步收紧思考长度限制，最终使模型学会了主动跳过冗余步骤且保证核心推理逻辑完整。

在自适应推理方向上，DeepSeek-V3.1[61]在同一

模型中同时支持思考模式与非思考模式，使模型能够根据任务复杂度和服务需求在快速响应与深度推理之间切换。简单问题可采用非思考模式以减少冗余思维链开销，复杂问题则启用思考模式以提升推理质量。这类方法将思维链压缩从单纯的长度缩短，进一步扩展为面向任务难度和服务约束的按需计算机制。

3.3 算子优化

Transformer 架构中的注意力机制因其计算复杂度随序列长度呈 2 次方增长，在长上下文场景下成为模型推理的主要瓶颈。尽管现有工作提出了多种稀疏化或线性化的替代架构，以降低或避免注意力机制的计算开销，但这些方法往往以损失模型精度为代价。因此，如何从计算与显存访问层面系统优化注意力机制的执行效率，成为当前重要的研究方向。从硬件执行层面分析，注意力计算的瓶颈往往并非浮点运算能力，而是受限于 GPU 显存与片上缓存之间的数据传输带宽，即所谓的显存墙问题。针对这一问题，文献 [24, 98−99] 提出了一系列注意力算子优化方法，旨在通过优化显存访问模式，在减少数据搬运的同时大幅提升计算吞吐。根据大模型推理过程中计算特性的不同，算子优化可分为针对计算密集型任务的预填充阶段优化，以及针对访存密集型任务的解码阶段优化。

3.3.1 预填充阶段优化

如图 12 所示，FlashAttention-V1[24] 针对预填充阶段提出了 I/O 感知的注意力算子，通过切块(tiling)技术将输入矩阵切分为能放入片上 SRAM 的小块，并在 SRAM 中融合了矩阵乘法与 Softmax 操作，避免了中间结果的显存读写。FlashAttention-V1 在算法层面的主要挑战在于 Softmax 的全局依赖性。为此，其引入了 Online Softmax 技术，通过维护并动态重缩放局部中间结果，实现了在 SRAM 内逐块计算注意力，有效解决了分块计算中的依赖问题。

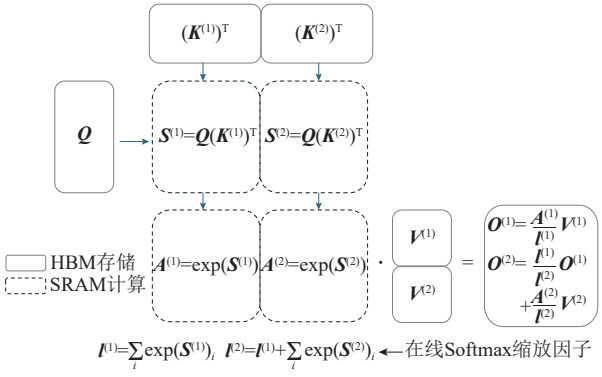


Fig. 12 Tiled computation and I/O-aware mechanisms in FlashAttention[98]

图 12 FlashAttention 中的分块计算与 I/O 感知机制[98]

然而，当处理长序列时，批次大小通常较小，导致可并行的线程块数量远低于 GPU 的 SM 计算单元数量，导致大量资源闲置。FlashAttention-V2[98] 针对该问题进一步优化了并行策略，在批次维度和注意力头维度之外，额外引入了序列长度维度的并行。同时 FlashAttention-V2 改进了线程块内部的工作划分，减少了非矩阵乘法运算开销，相比于 FlashAttention-V1 实现了 2 倍的加速。FlashAttention-V3[99] 则针对于 Hopper 架构优化了 warp 调度，实现生产者与消费者分离，实现异步数据搬运与计算的流水线重叠，同时支持 FP8 低精度硬件加速，进一步释放了硬件的峰值性能。

3.3.2 解码阶段优化

在预填充阶段，FlashAttention 会根据批次数量和查询(query)长度进行并行化。但在解码阶段，由于每次仅生成一个词元，因此查询长度通常为 1，且批处理数量通常较小，直接使用 FlashAttention 会导致 GPU 计算单元利用率严重不足。针对这一问题，FlashDecoding[25] 提出在键值对的序列长度维度上进行并行化。如图 13 所示，FlashDecoding 将键值缓存切分为多个块，并行计算每个块的注意力结果，最后通过一个规约步骤得到注意力结果。这种方法在长上下文推理场景下，能够充分利用 GPU 资源，相比传统方法实现了高达 8 倍的加速。

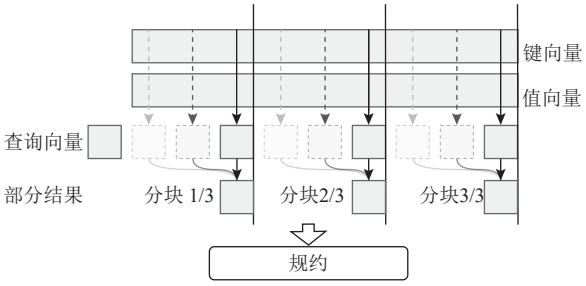


Fig. 13 Split-KV parallelism computing strategy in FlashDecoding[25]

图 13 FlashDecoding 中的键值分块并行计算策略[25]

尽管 FlashDecoding 解决了并行度问题，但在处理 Softmax 结果时仍需同步更新。FlashDecoding++[100] 对此进行了改进，通过统计规律预估一个统一的缩放因子，使得各个部分 Softmax 计算可以独立进行，减少了同步等待的成本。另外，FlashDecoding++针对解码阶段常见的扁平 GEMM(即低维矩阵乘法)进行了优化，利用双缓冲技术和更精细的填充策略解决

了计算资源利用率低的问题，并引入了启发式数据流以适应不同硬件配置，从而在多种 GPU 平台上实现了显著的性能提升。

4 服务层优化

大模型正逐渐从实验室的研究对象转变为大规模生产力工具，其推理系统的设计理念也发生了显著变化。早期的推理引擎主要关注于单个请求的计算延迟优化，而目前推理系统已演变为包含显存管理、请求调度、异构卸载等功能的复杂平台，以在延迟、吞吐量、服务质量之间寻找最优权衡。本节将梳理大模型推理相关引擎的演进脉络，如表 3 所示，包括 FT（faster transformer）[26]，TensorRT-LLM[27]，vLLM[28]，SGLang[29]，TGI（text generation inference）[30]，llama.cpp[31]，KTransformers[32]，Moomcake[101]。随后从请求调度算法、PD 分离以及异构卸载这 3 个核心维度分析现有工作。

Table 3 Feature Comparison of Mainstream LLMs Inference Platforms

表 3 主流大模型推理平台特性对比

特性	TensorRT-LLM[27]	vLLM[28]	SGLang[29]	Text Generation Inference[30]	Llama.cpp[31]	KTransformers[32]	Mooncake[101]
核心定位	极致性能、生产环境、NVIDIA 生态	易用、生态兼容性好	复杂逻辑、结构化、高吞吐	生产级部署、安全性、稳定性、HF 集成	个人 PC、端侧设备、广泛硬件兼容	异构计算、消费级显卡运行大 MoE 模型	以键值缓存为中心的大模型推理服务后端平台
显存管理	分页 KV 缓存	PagedAttention	RadixAttention	PagedAttention	环形缓存区	基于 SGLang	分布式键值缓存
调度算法	连续批处理	FCFS 和抢占	最长公共前缀匹配	连续批处理	简单批处理	异构流水线并行	以键值缓存为中心的全局调度
PD 分离	支持（通过 Triton 编排）	支持（NIXL/P2PNCCL）	支持（NIXL/Mooncake）	依赖外部编排	不支持	不支持	原生支持
异构能力	仅存储卸载	仅存储卸载	仅存储卸载	无	存储卸载和计算卸载	存储卸载和计算卸载	存储卸载

4.1 推理引擎演进脉络

大模型推理的自回归特性决定了其解码阶段是典型的显存受限，而预填充阶段则是计算受限。因而大部分推理引擎优化都围绕着如何提高显存带宽以及计算单元利用率展开。

FT[26] 作为早期推理优化的奠基者，针对 Transformer 架构引入了层融合技术，将多头注意力中的矩阵乘法、偏置加法及激活函数合并到 1 个算子中，显著减少了显存读写次数，确定了高性能 Transformer 内核实现的标杆。但 FT 诞生于静态批处理时代，在处理一个批次的请求时，必须等待批次内最长序列完成生成，短序列在结束后需要填充（padding）到与最长序列相同长度。上述计算要求导致了极大的显存资源和计算资源浪费，无法适应请求长度差异巨大的真实服务场景。

TensorRT-LLM[27] 在 FT 的基础上针对动态负载进行了优化，引入了连续批处理（in-flight batching）技术，允许请求的动态插入和退出，极大提高了 GPU 的利用率。此外，TensorRT-LLM 支持将 Python 语言转化为高效的内核代码，能够充分挖掘硬件性能，实现高吞吐与低延迟的推理服务。

vLLM[28] 由 UC Berkeley 团队提出，旨在解决静态显存分配导致的显存碎片化问题。vLLM 根据操作系统中虚拟内存的分页思想，提出了 PagedAttention 显存管理机制。传统的键值缓存需要预先分配一块连续的显存空间，其大小通常由模型最大的输入和输出长度决定，以确保模型在推理时不会出现显存溢出。但这种分配模式导致显存利用率极低，大部分请求都无法占满其显存空间。PagedAttention 将键值缓存切分为固定大小的块，并维护一个逻辑块到物理块的映射表，允许键值缓存在物理显存中非连续存储。这一机制极大减少了显存碎片，能够支持更大的批处理，从而大幅提升了系统吞吐量。

随着智能体和多轮对话应用的兴起，请求之间往往存在大量的公共前缀，比如系统提示词或少样本示例。vLLM 虽然解决了显存分配问题，但是在处理这种结构化复用时仍然需要重新计算前缀的键值缓存。针对上述问题，SGLang[29] 提出了 RadixAttention，将键值缓存组织为 1 棵基数树（radix tree）而非独立的请求队列。树结构天然保留了生成后的分支结构，因而 SGLang 在运行时能够自动识别新请求与历史请求的公共前缀，并直接复用显存中的键值缓存，无需再次计算。

TGI[30] 是由 Hugging Face 推出的生产级服务框架，其重点在于生产环境的稳定性与易用性。TGI 最早引入了 Rust 编写的调度层并实现了连续批处理。调度层负责处理高并发的 HTTP 请求、队列管理和批处理编排，而 Python 端则专注于模型计算，从而解决

了 Python 在处理高并发网络 I/O 时的性能瓶颈。同时 TGI 集成了 Safetensors 安全加载、水印等功能，解决了企业在将大模型转化为 API 服务时面临的安全性与合规性问题。

llama.cpp[31] 与 KTransformers[32] 均致力于在硬件资源受限的环境中高效运行大模型。llama.cpp 面向消费级设备显存与算力有限的特点，设计了 GGUF 量化格式与 *K*-Quants 量化技术，并支持模型的层级卸载（layer-wise offloading）。该框架采用底层语言实现以降低运行时开销，并针对苹果芯片及多种 CPU 指令集（如 AVX/AMX/NEON）进行了深度优化。其层级卸载机制可将模型部分层加载至 GPU 显存执行计算，其余层则保留在 CPU 内存中运行。KTransformers 则针对混合专家模型（MoE）激活参数量少但总参数量大的特性，设计了专为 MoE 优化的极致异构卸载架构：将注意力层及部分专家常驻 GPU，其余专家置于 CPU 进行计算，实现了 CPU 与 GPU 间的流水线并行。目前，KTransformers 已与 SGLang 集成，用于实际生产部署。

除上述端到端推理引擎外，近年来一批以键值缓存为中心的推理服务平台在生产环境中获得了广泛关注，其中 Moonshot AI 推出的 Mooncake[101] 是该类工作的代表。与上述引擎定位不同，Mooncake 并不独立承担模型计算与内核实现，而是聚焦于 PD 分离架构、分布式键值缓存存储、跨实例键值传输以及缓存感知调度等系统级协同能力，通常与 vLLM，SGLang 等推理引擎协同部署，以支撑超大规模在线服务场景。关于 Mooncake 的具体设计将结合 PD 分离架构和异构卸载展开详细讨论。

随着推理引擎的不断迭代，各大系统积极集成社区涌现的最新优化技术，并在架构与策略上相互借鉴，使得彼此在功能与性能上的差异逐渐收敛。下文将聚焦推理系统中的关键技术，对请求调度算法、PD 分离架构以及异构卸载方案的主流实现进行详细分析。

4.2 请求调度算法

在大模型推理系统中，调度策略极为关键，其好坏直接决定了计算资源的分配效率与服务质量。传统静态批处理因不同请求序列长短不一，整个批次必须等待最长序列生成完才能统一返回。这一特点会导致短序列请求在生成结束后仍长时间占用显存，且无法插入其他的新请求，造成资源浪费与延迟增加，即存在队头阻塞问题（head-of-line blocking）。Orca[102] 对此提出了连续批处理，其核心思想是将调度粒度从请求级细化到迭代级，在模型每迭代一步之后检查请求状态，允许已完成的请求立即退出并释放资源，同时动态插入新请求填补空缺。连续批处理显著提升了推理系统的吞吐量与有效资源利用率。

虽然连续批处理解决了队头阻塞问题，但在混合处理预填充与解码阶段时，仍面临计算利用率不均匀挑战。如图 14(a)所示，当一个长序列的预填充任务请求加入时，会长时间占用 GPU 的计算单元，导致正在进行解码的任务挂起。针对这一问题，Sarathi-Serve[103] 提出了分块预填充技术。如图 14(b)所示，Sarathi-Serve 将长提示词序列拆分为多个小块，利用解码阶段的算力空闲间隙，将这些预填充块搭载在解码批次中流水线执行计算。通过这种方式，Sarathi-Serve 构建的混合批次具有近似均衡的计算需求，将预填充对解码阶段的词间延迟（time between tokens, TBT）的影响降低到了最低。目前连续批处理与分块预填充技术已经成为了上述推理引擎的事实标准。

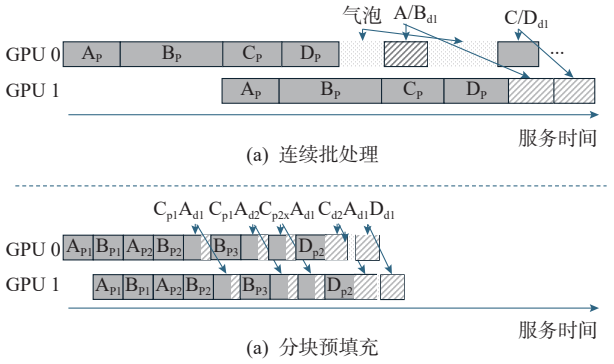


Fig. 14 Illustration of Continuous batching and chunked prefill[103]

图 14 连续批处理与分块预填充示意图[103]

在上述 2 种调度机制的基础上，不同推理引擎做了不同的优化。vLLM 采用先来先服务（first come first serve, FCFS）策略，其调度器会检查当前 GPU 显存分配器中的空闲物理块数量是否支持运行队列中的请求生成下一个词元。当显存不足时，vLLM 会触发抢占机制，将部分低优先级的键值缓存换出到 CPU 内存，待显存充裕后再换回 GPU 显存中。TGI 的调度算法同样支持基于优先级的请求调度、请求排队和超时处理，并内置自动扩缩容机制，使其更适应生产环境的 SLO 要求。除此之外，TGI 在构建批次时，会根据提示词的长度和生成参数对请求进行分组，以优化批次的形成。这种启发式策略倾向于将相似特征的请求聚合，从而提升批次内计算的均衡性。SGLang 的调度器则将请求组织为 1 棵基数树，

优先调度最长前缀匹配的请求。在多轮会话或智能体场景下，这种机制能够显著提升吞吐量。llama.cpp与KTransformers则关注于资源受限场景的推理服务，通常处理低并发请求，其请求调度策略相对简化，主要专注于异构计算资源的分配与显存利用率的极致优化。

4.3 PD分离

PD分离的核心思想是将预填充和解码阶段解耦，部署到独立的GPU资源池中。预填充实例负责计算提示词并生成键值缓存，然后通过高速网络将键值缓存传输给解码实例，解码实例逐个生成词元。其主要目的是消除阶段间干扰，天然解决了混合部署导致的队头阻塞问题，且可以根据两者的特性独立配置资源和优化手段，从而最大化系统的吞吐量。本节主要讲述各PD分离架构所采取的不同技术路径和权衡。

DistServe[104]系统性地分析了PD混合推理架构产生的问题，并通过实验发现预填充阶段适合采用张量并行TP，将单个大计算任务拆分到多个GPU上协同完成以聚合算力，而解码阶段则适合采用流水线并行(pipeline parallel, PP)，让多个批次在流水线的不同阶段上重叠执行。为了解决PD分离架构中键值缓存传输开销大的挑战，DistServe设计了带宽感知的放置算法，将大模型按层划分为多个阶段。每个阶段的预填充和解码实例放置在同一节点上，阶段之间则采用流水线并行，仅需传输每个词元的中间层激活值。此外，DistServe通过模拟器确定预填充和解码实例的最佳配置，从而显著提高了推理系统的吞吐量。

DistServe找到最优配置后，其实例的分布情况不再发生变化，这种设计在面对动态负载时会导致阶段间负载不匹配。例如预填充计算饱和而解码闲置，或解码资源紧张而预填充空闲。SplitWise[105]提出了一种自适应推理架构以解决上述问题。该架构引入了预填充、解码、PD混合3种资源池，预填充和解码资源池专注于执行对应阶段的计算任务。当负载发生剧烈变化时，由PD混合资源池来完成计算任务。SplitWise同时构建了集群级调度器CLS和机器级调度器MLS。其中CLS负责基于负载感知的全局路由与粗粒度资源分配，将请求分发到对应的机器上，MLS则使用先到先服务调度算法处理本地请求，并支持优先级调度。在键值缓存的传输方面，SplitWise设计了逐层传输机制，一旦模型的某一层完成计算，立即将该层的键值缓存传输到解码实例中，从而实现计算与通信重叠，有效地掩盖了跨节点的通信延迟。

ShuffleInfer[106]在PD分离的架构基础上，允许不同实例根据负载变化进行角色互换，确保系统在资源分配方面更加灵活高效。为了消除角色互换时的资源争抢与延迟，系统采用了分块预填充机制，严格限制单次迭代处理的词元数量，使计算过程维持在计算与显存带宽利用率的平衡点。在调度层面，ShuffleInfer引入了全局和本地2级调度器，全局调度器负责宏观的请求分发与资源角色管理，而位于预填充实例内部的本地调度器则基于输出长度的预测结果来调度解码阶段的任务，支持最短作业优先等调度算法，能够主动规避调度热点的形成，提升了系统在混合工作负载下的稳定性和吞吐量。

P/D-Serve[107]致力于解决PD分离架构在万卡GPU的大规模集群中面临的工程通信与调度瓶颈。针对大规模RDMA网络中，传统按块传输效率低下的问题，P/D-Serve在发送端将不连续的键值块重组为连续缓冲区，并在接收端通过RecvScatter算子还原，从而减少了设备间的通信次数并提升了带宽利用率。为了解决负载不均匀的问题，P/D-Serve引入了一种按需转发机制，并且能够根据请求的性质和流量变化动态调整预填充和解码实例的比例。当预填充实例拒绝请求或处于忙碌状态时，网关能够将请求转发到空闲实例上，从而避免请求超时。为了提升系统稳定性，P/D-Serve实现了自动健康监测与故障检测机制，能够在发现故障后以最低的成本自动替换故障实例。

Mooncake[101]是Moonshot AI服务的后端平台。为了充分利用各种硬件资源，Mooncake将物理资源重构为3个独立伸缩的资源池，包括预填充资源池、解码资源池以及利用存算分离架构构建的分布式键值缓存池。在键值缓存的传输机制上，Mooncake采用了与上述Splitwise相同的逐层传输机制，实现了计算与传输的异步流水线，有效掩盖了通信延迟。此外，Mooncake采用前缀匹配和缓存迁移技术进行复用，并支持冷热数据分离的分层存储策略，热点数据常驻显存或CPU内存，冷数据则下沉至SSD。在请求调度方面，Mooncake设计了一个全局调度器，负责基于前缀缓存命中率和各节点实时负载进行路由决策。同时利用预测模型，在处理请求前预估端到端延迟，提前拒绝那些无法满足服务水平目标的请求，避免无效计算带来的资源浪费。

从上述工作可以发现，PD分离系统在实际部署中始终围绕2个核心问题展开：一是如何确定预填

充与解码实例的最优比例以充分利用集群算力;二是如何控制键值缓存的跨实例传输开销以避免通信反成瓶颈。对前者,在稳态负载下可通过匹配 2 个阶段吞吐的方式得到直观的解析关系。记请求平均输入长度为L、输出长度为G,单个预填充请求的时延为$T_{\mathrm{p}}(L)$、单个解码步的时延为T_{d},则 2 个阶段的最大吞吐量分别为$\mu_{\mathrm{p}}=N_{\mathrm{p}}/T_{\mathrm{p}}(L)$与$\mu_{\mathrm{d}}=N_{\mathrm{d}}/(G\times T_{\mathrm{d}})$。令$\mu_{\mathrm{p}}=\mu_{\mathrm{d}}$,则最优实例比例如式(9)所示:

$$\frac{N_{\mathrm{p}}^{*}}{N_{\mathrm{d}}^{*}}=\frac{T_{\mathrm{p}(L)}}{G\times T_{\mathrm{d}}}\text{。} \tag{9}$$

上述内容表明,预填充和解码的最优比例仅由输入和输出长度比例与单阶段时延决定,长输入短输出场景(如检索增强生成、长文档问答)应倾斜更多预填充实例,而短输入长输出场景(如数学推理、长思维链生成)则需增加解码实例。这也正是 DOPD[108] 等工作在动态负载下执行弹性伸缩的理论依据。

对后者,PD 分离的通信和计算权衡可由键值缓存传输开销与预填充计算时延的比例来量化。设模型参数量为N_{params},层数为N_{layer},单个词元每层键值大小为s_{kv}字节,预填充实例算力为C_{p},实例间有效通信带宽为BW。则单个请求的键值缓存总量为$K=N_{\mathrm{layer}}s_{\mathrm{kv}}L$,传输时延$T_{\mathrm{comm}}=K/BW$,预填充时延近似为$T_{\mathrm{p}}\approx 2N_{\mathrm{params}}L/C_{\mathrm{p}}$。令 PD 分离保持净收益的条件为$T_{\mathrm{comm}}\leqslant\beta T_{\mathrm{p}}$(其中$\beta\in(0,1)$为通信开销容忍因子),临界带宽如式(10)所示:

$$BW_{\mathrm{crit}}=\frac{N_{\mathrm{layer}}s_{\mathrm{kv}}C_{\mathrm{p}}}{2\beta N_{\mathrm{params}}}\text{。} \tag{10}$$

从式(10)可以看出,在理想情况下,BW_{crit}与输入长度L无关,仅由模型结构与算力-带宽比决定。但实际部署中,PD 分离场景还会受到请求到达的突发性、变长负载混合、网络竞争导致的带宽波动、批处理内填充开销等诸多因素影响,实际部署中仍需结合真实环境和负载进行修正。

当前主流推理引擎也基于 PD 分离架构提供了不同的设计。例如,SGLang 维护了预填充和解码资源池,两者可以根据负载需求独立扩展并使用不同的并行度设置,并采用缓存感知路由策略,优先将请求发送到最长公共前缀的实例,从而最大化键值缓存的复用。目前 SGLang 支持 Mooncake 和 NIXL 作为键值传输引擎。vLLM 的 PD 分离尚处于实验阶段,仅静态地分配预填充和解码实例,支持 LMCache, NIXL, P2PNCCL 等键值传输引擎。TensorRT-LLM 主要通过 Triton Server 进行编排,利用 NVLink 或 RDMA 进行键值缓存的高速传输。TGI 目前关注单节点优化,更多依赖于外部编排系统来实现 PD 分离。

4.4 异构卸载

大模型的庞大规模使其参数量极大,直接完全加载至 GPU 需要消耗大量昂贵的显存资源。为此,异构卸载已成为降低部署门槛的关键策略。异构卸载技术主要包含 2 个维度的优化:一是存储卸载,即利用系统内存与 SSD 实现显存扩展,突破显存物理上限;二是计算卸载,即利用 CPU 的异构算力实现协同计算,以进一步提升系统吞吐量和资源利用率。

在存储卸载方面,随着模型规模、上下文长度和服务并发度持续增长,其卸载对象已从模型权重扩展到推理过程中产生的键值缓存、多轮会话状态与中间激活等多类数据。相应地,推理存储架构正从单一的 GPU 显存管理向 GPU 显存、CPU 内存、本地 SSD 与远程存储协同的多级异构体系演进。对于模型权重,关键问题是在有限显存下控制模型加载与层间搬移开销,尤其需要降低实例启动和弹性扩缩容过程中的权重加载时延。DeepSpeed 提出的 ZeRO-Inference[109] 将模型权重存储在内存或固态盘中,逐层加载模型参数至 GPU 进行计算,并通过权重预取尽可能掩盖异构介质之间的数据传输时延。目前,vLLM, SGLang 等推理框架同样集成了类似的权重卸载功能。

相较于访问模式相对稳定的模型权重,键值缓存及其索引元数据会随输入和生成序列动态增长,是长上下文数据管理和多轮会话状态维护的主要对象。在长上下文与高并发场景下,系统不仅需要解决存储容量快速增长的问题,还需协调跨请求复用、跨会话持久化、多级存储调度以及并发数据加载之间的关系。针对多轮对话中键值缓存的跨会话持久化问题,CachedAttention[110] 构建了基于内存和 SSD 的层次化存储系统,通过层级预取、异步写回与解耦位置编码实现跨会话的键值复用,显著降低了首词元延迟。FlashGen[111] 在类似的 GPU/CPU/SSD 多级缓存架构上进一步关注高并发访问下的数据调度,通过请求重排序,缓解 FCFS 调度下长提示词引起的队头阻塞,从而提高多轮对话场景的系统吞吐量。除完整保存键值缓存外,HCache[112] 观察到在标准多头注意力中,单层隐藏状态所占空间为键值缓存的一半,因此持久化保存隐藏状态,并在恢复时通过轻量计算重建键值缓存,从而在存储开销与重计算成本之间取得平衡。

在集群级 PD 分离架构下,上述数据的跨节点加

载还可能形成 I/O 与网络带宽瓶颈。DeepSeek 提出的 DualPath[113] 发现 PD 分离部署中预填充实例存储网卡常被打满，而解码实例网卡长期闲置，导致集群带宽浪费。如图 15 所示，DualPath 在传统的持久化存储到预填充实例的路径之外，额外引入了持久化存储到解码实例再到预填充实例的 RDMA 路径，从而利用闲置的解码侧存储网卡带宽完成键值加载，使集群存储带宽利用率近乎翻倍。Mooncake[101] 与 LMCache[114] 则进一步从全局视角将键值缓存作为集群共享资源进行管理。其中，Mooncake 通过存算分离架构构建跨节点的分布式键值缓存池，并集成前缀匹配、冷热数据分层和全局调度机制，根据缓存命中情况与节点负载协调高并发请求的数据访问。LMCache 作为可插拔的键值缓存中间层，为 vLLM, SGLang 等推理引擎提供多级异构存储方案。HiCache 则与 SGLang 自身的基数树缓存深度耦合，在多级存储中保持树状前缀索引结构，实现键值缓存的跨层迁移与高效复用。总体而言，异构存储面临的核心问题已由单纯扩展容量转变为对容量、带宽、访问时延和数据复用效率的协同优化：系统既要依据数据热度与生命周期完成分层放置、预取和迁移，也要在高并发条件下协调缓存淘汰、数据加载与请求调度。

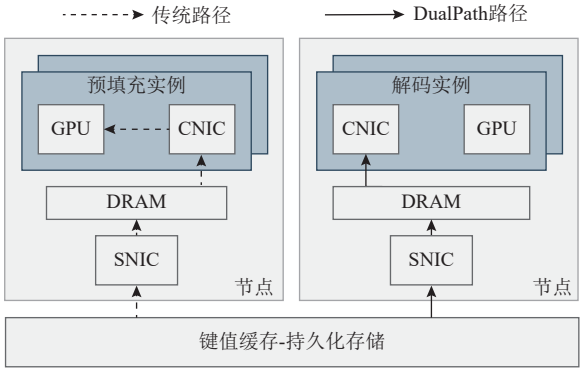


Fig. 15 Dual-path loading mechanism of DualPath[113]

图 15 DualPath 双路径加载机制[113]

在计算卸载方面，llama.cpp[31] 利用 AVX, NEON, AMX 指令集对异构计算进行了深度优化，设计了高效的底层算子与 GGUF 量化格式，显著加速了量化模型的推理过程。同时，llama.cpp 提供了适配不同硬件平台的推理内核，采用即时反量化策略，即在寄存器层面将低比特量化数据解压并计算，从而在大幅降低显存带宽占用的同时，实现了接近原始精度的推理性能。在存储卸载策略上，llama.cpp 采用层级卸载，将模型的前若干层放置在 GPU 中计算，其余层在 CPU 上进行计算。

KTransformers[32] 则是为混合专家模型（MoE）设计的推理系统，能够在单张消费级显卡上流畅运行千亿参数级别的模型。其核心设计充分结合了 MoE 的稀疏特性，通过异构计算卸载策略实现高效推理。如图 16 所示，KTransformers 将每层的共享专家和注意力部分固定在 GPU 计算，而将稀疏激活的专家放置在 CPU 内存中并按需计算，既能显著降低显存需求，也契合了 MoE 稀疏激活的特性。为了进一步提升 CPU 利用率和推理速度，系统引入了专家推迟机制，允许将一部分路由专家的计算延迟至下一层执行，从而实现 CPU 与 GPU 的高效流水线并行。实验分析表明，异构推理的瓶颈不在于 CPU-GPU 间的 PCIe 传输带宽，而是频繁启动内核带来的开销。为此，KTransformers 将整个解码阶段封装为单一的 CUDA Graph 实例，极大降低了内核启动开销。

4.5 性能测试

本文评估了主流推理引擎在不同计算环境下的性能表现。根据应用场景与硬件资源的差异，本文将评估对象分为 3 类：一是追求高吞吐量与低延迟的

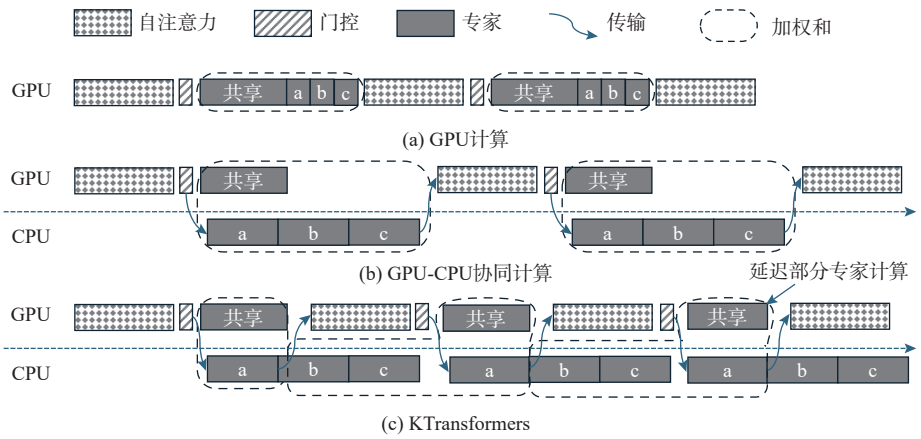


Fig. 16 Heterogeneous Offloading strategy in KTransformers[32]

图 16 KTransformers 异构卸载策略[32]

推理引擎, 包括 vLLM, SGLang, TensorRT-LLM, TGI。由于 TGI 推理引擎专为在线服务场景设计, 因此本文仅测试其在线使用场景的性能; 二是面向资源受限的消费级硬件的推理引擎, 包括 KTransformers 和 llama.cpp, 主要关注基于 CPU-GPU 异构卸载的单机推理能力; 三是键值缓存卸载和复用效率对比, 分析不同引擎设计的性能差异。

为了覆盖不同的计算需求, 本文选取了具有代表性的 Qwen2.5-3B, Qwen2.5-32B, Qwen3-30B-A3B 模型进行测试, 其中 Qwen2.5-3B 和 Qwen3-30B-A3B 部署于单卡, 且 Qwen3-30B-A3B 会将部分权重卸载到内存中, Qwen2.5-32B 则采用 8 卡张量并行。实验环境与推理引擎版本如表 4 所示。

Table 4 Parameters of Experimental Environment

表 4 实验环境参数

名称	参数/版本
CPU	Intel(R) Xeon(R) Platinum 8360Y
GPU	8 卡 4090/24 GB
内存/GB	256
vLLM	0.13.0
SGLang	0.5.6.post2
TGI	2.0.5.dev0
TensorRT-LLM	1.0.0
llama.cpp	cef1d23
KTransformers	0.4.3
LMCache	0.4.2

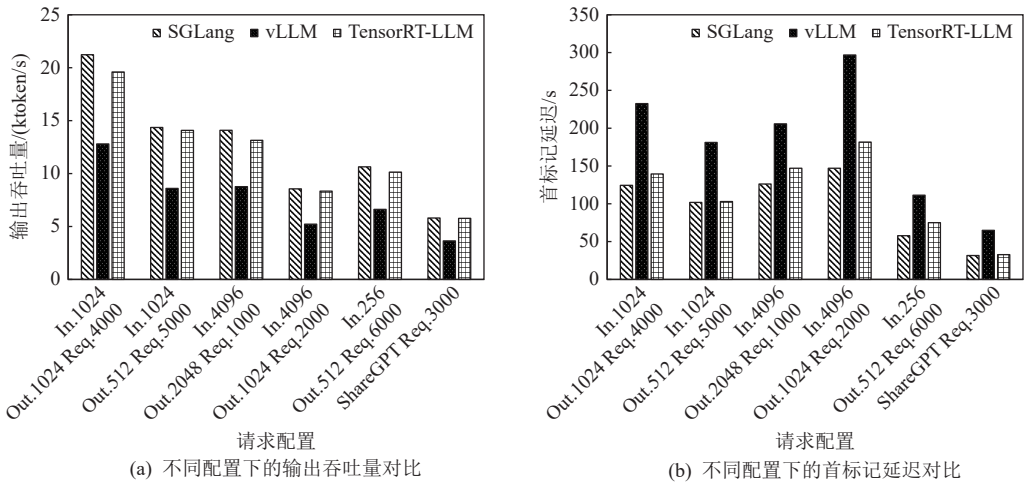
测试集包括了合成数据集和 ShareGPT 数据集。合成数据集能够通过参数精确控制输入和输出长度, 测试推理引擎的极致性能。ShareGPT 是目前大模型推理引擎测试中最常用的标准数据集, 源自用户与 ChatGPT 的真实对话记录, 其输入长度不一, 且输入和输出长度的比例变化较大, 能够有效模拟生产环境中的真实流量分布。

针对离线场景, 一次性发送 1 000~6 000 个请求, 重点评估系统的输出吞吐量以及首令牌延迟, 测试集包括合成数据集和 ShareGPT 数据集。针对在线场景, 则以 1~16 个/s 请求的速率发送请求, 主要测试推理引擎在不同并发下的表现。为了消除序列长度的干扰, 在线场景仅采用合成数据集, 模型的输入输出均设置为 1 024, 严格控制请求发送的速率。

图 17 和图 18 展示了离线使用场景中不同模型参数大小下, 各推理引擎的输出吞吐量和首词元延迟。In., Out., Req.分别表示输入长度、输出长度以及请求数量。对于 Qwen2.5-3B 模型, SGLang 和 TensorRT-LLM 在输入较短的数据集上均能实现 5 000 token/s 以上的吞吐量, 且 SGLang 略优于 TensorRT-LLM。相比之下, vLLM 在该场景表现出一定的性能瓶颈。在输入长度 256 与输出长度 512 的测试配置下, vLLM 的吞吐量仅为 SGLang 的 62.26%。对于首词元延迟, vLLM 也远高于 SGLang 和 TensorRT-LLM。

随着模型规模扩展至 Qwen2.5-32B, 推理任务的计算密度显著增加, GPU 需要花费更多的时间在矩阵乘法上, 推理引擎本身的调度开销在总时间里的占比减少, 因此不同框架之间的差距逐渐缩小。此外, 由于采用了张量并行, 引入了额外的跨 GPU 通信开销。而通信速度受限于 PCIe 带宽, 进一步掩盖了软件层面的差异。

对于在线使用场景, 本文侧重于观察不同请求速率条件下, 各推理引擎的端到端延迟。如图 19 所

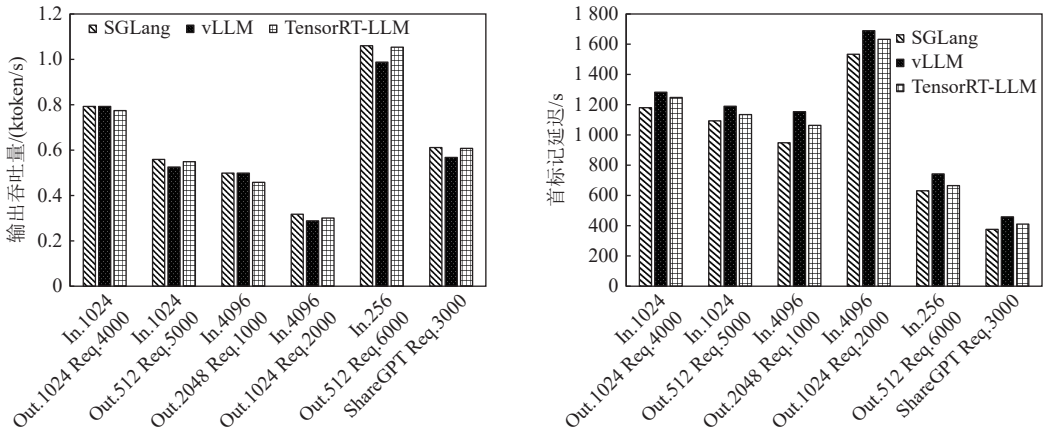


(a) 不同配置下的输出吞吐量对比

(b) 不同配置下的首标记延迟对比

Fig. 17 Comparison of inference performance of Qwen2.5-3B model under different inference frameworks and request configurations

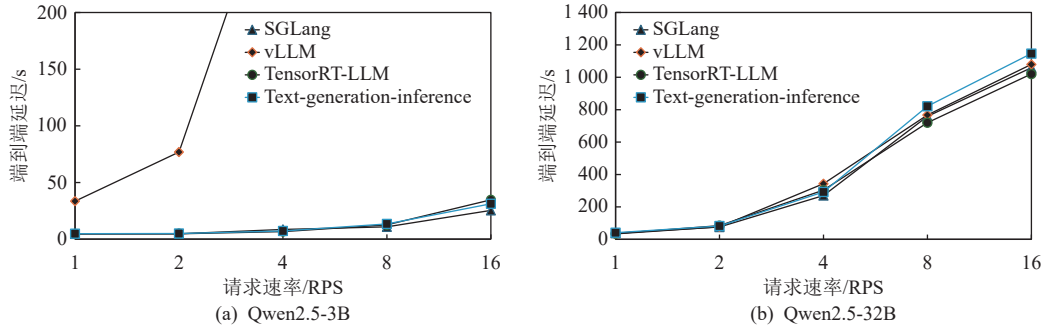
图 17 Qwen2.5-3B 模型在不同推理框架及请求配置下的推理性能对比



(a) 不同配置下的输出吞吐量对比 (b) 不同配置下的首标记延迟对比

Fig. 18 Comparison of inference performance of Qwen2.5-32B model under different inference frameworks and request configurations

图 18 Qwen2.5-32B 模型在不同推理框架及请求配置下的推理性能对比



(a) Qwen2.5-3B (b) Qwen2.5-32B

Fig. 19 Comparison of End-to-end latency of different inference frameworks on Qwen2.5-3B and Qwen2.5-32B under different request rates

图 19 不同请求速率下各推理框架在 Qwen2.5-3B 和 Qwen2.5-32B 上的端到端延迟对比

示，SGLang, TensorRT-LLM, TGI 在小型模型下性能相当，其延迟保持在 50 s 内，而 vLLM 在 RPS≥3 时延迟显著增加。对于 Qwen2.5-32B 模型，其趋势与离线场景类似，各推理引擎的性能差异被 GPU 的计算与通信开销磨平。

与上述追求极致吞吐量的服务端引擎不同，llama.cpp 和 KTransformers 倾向于在个人 PC 上部署大模型服务，其测试重点在于不同方式的异构卸载策略对于性能的影响。因此本实验将请求数统一设定为 2，模型采用了 Qwen3-30B-A3B，并在单卡环境上进行测试。对于 llama.cpp，采用了 4b 量化后的 GGUF 格式。实验结果如图 20 所示，KTransformers 的输出吞吐量均显著优于 llama.cpp，实现了约 1.9~3.1 倍的性能提升。其核心原因在于 llama.cpp 采用了层级卸载策略，层与层之间的计算存在串行等待，且没有针对 MoE 的稀疏性做特殊拆分。KTransformers 则将模型中所有注意力层、非专家组件以及小部分专家驻留于 GPU 显存中执行，而将占参数量主体的其余专家权重卸载至 CPU 内存中执行。这种架构使得 2 部分计算资源能够以流水线方式协同工作，从而在整体上显著提升了资源利用率与推理效率，相比于层级串行卸载的 llama.cpp 展现出明显的性能优势。

上述实验聚焦于 CPU-GPU 异构卸载场景下的

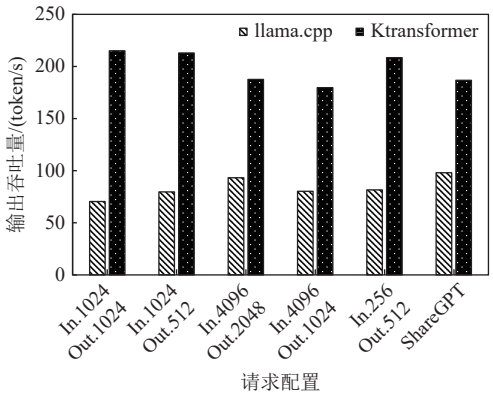


Fig. 20 Comparison of output throughput of KTransformers and llama.cpp

图 20 KTransformers 与 llama.cpp 的输出吞吐量对比

吞吐对比。除计算卸载外，键值缓存的存储卸载能够扩展显存，为前缀缓存复用提供更大的空间，是降低首词延迟 TTFT 的关键因素。为此，本节进一步比较 vLLM 与 SGLang 在 GPU–CPU 内存–SSD3 级存储下的键值缓存复用效率，两者分别使用 LMCache 和 HiCache 实现键值缓存卸载，但采用的加载策略并不相同。vLLM + LMCache 提供同步与异步 2 种加载方式，默认配置为同步加载，SGLang + HiCache 则提供 best_effort, timeout, wait_complete 这 3 种预取策略。其中，best_effort 为默认配置，仅利用调度时已经位于 CPU 内存中的键值缓存，若 SSD 预取尚未完成则放弃该部分复用；wait_complete 则等待所需键值缓存全部从 SSD 预取到 CPU 内存后，再执行后续的内存到 GPU 加载与计算；timeout 则设定了 SSD 读取时间，完成或超时后停止 SSD 读取。

实验使用 Qwen3-8B 模型，部署在单卡 4090 上，CPU 内存配置为 24 GB，SSD 容量配置为 200 GB。测试负载采用长文档问答（long-doc QA）和多轮对话（multi-turn）。前者包含 46 篇约 10 000 词元的文档，并重复 3 轮，用于模拟相同长文档前缀被跨请求反复访问的场景，后者包含 80 个并发会话和 10 轮对话，统计前 400 个完成请求，用于模拟多轮交互中上下文逐轮增长的场景。

如图 21 所示，不启用键值缓存卸载时，vLLM 与 SGLang 在长文档测试负载下的复用率均接近 0，TTFT 分别为 2.44 s 和 3.10 s。启用 LMCache 后，vLLM 在同步与异步 2 种加载方式下都能达到 74.6% 的复用率，但异步加载将 TTFT 从 1.55 s 进一步降至 0.95 s，说明 SSD 到 CPU 的读取开销被有效隐藏在异步预取过程中。相比之下，SGLang+HiCache 的默认 best_effort 策略几乎无法利用 SSD 中的键值缓存，复用率仅 0.6%，TTFT 为 3.19 s。其原因在于长文档前缀较大，SSD 预取通常无法在单次调度窗口内完成，而 best_effort 在数据尚未到达 CPU 内存时会直接放弃复用。将策略切换为 timeout 或 wait_complete 后，调度器为 SSD 预取留出等待窗口，复用率分别提升至 45.4% 和 34.6%，TTFT 也分别降至 1.15 s 和 1.16 s。

图 22 展示了多轮对话测试负载下的对比结果。vLLM+LMCache 仍然表现稳定，异步加载和同步加载的命中率均为 65.0%，TTFT 分别为 1.30 s 和 1.50 s，说明 LMCache 在多轮增量前缀场景中也能稳定获得缓存收益。SGLang+HiCache 在该负载下对预取策略更加敏感，timeout 能取得 53.7% 的命中率和 1.76 s 的 TTFT，是 3 种预取策略中最均衡的配置，默认 best_effort 策略由于不等待 SSD 预取完成，命中率降至 23.2%，TTFT 为 2.03 s。而 wait_complete 虽然命中率最高，但 TTFT 升至 7.12 s。其原因在于多轮对话测试负载具有更高并发度，多个会话会同时挂起等待 SSD 预取，导致磁盘带宽被大量请求分摊，进而拖慢调度流水线。因此，较高的命中率并不必然意味着较低的 TTFT，在高并发场景中，过度等待 SSD 反而会使缓存复用收益被 I/O 阻塞抵消。

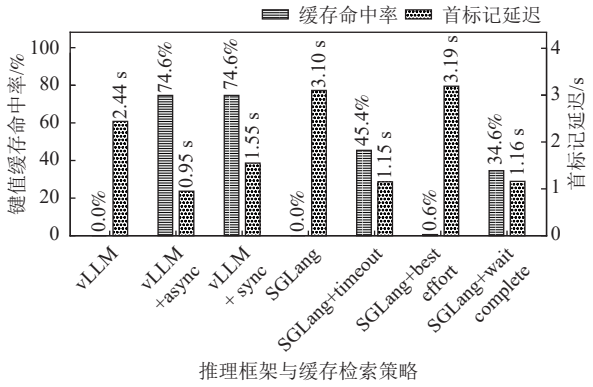


Fig. 21 Comparison of KV cache reuse rate and TTFT in long-document QA

图 21 长文档问答中键值缓存复用率与首词延迟对比

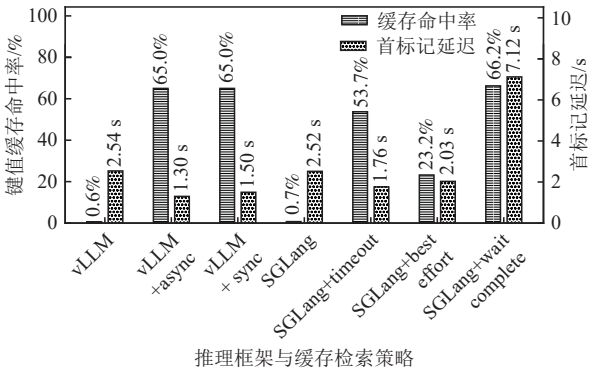


Fig. 22 Comparison of KV cache reuse rate and TTFT in multi-turn dialogue

图 22 多轮对话下键值缓存复用率与首词延迟对比

5 总结与展望

本文从模型层、算法层和服务层 3 个维度，系统性地梳理了大模型推理优化的关键技术与研究进展。尽管现有工作已取得了显著成效，有效缓解了显存墙与计算墙的限制，但各项技术均存在特定的适用边界，其实际收益高度依赖于工作负载特征、模型规模与硬件条件，也伴随着不同程度的精度损失或工程开销。因此，本节首先梳理现有优化技术的主要局限性与适用场景，并探讨各技术路径之间的组合

优化与协同关系，最后展望未来的重点研究方向。

5.1 现有技术的局限性与适用场景

在模型层，量化技术虽能显著降低显存与带宽开销，但低比特量化在 1B 参数模型上会出现精度崩溃现象，且部分早期工作在运行时引入了动态离群值检测等额外计算，其实际推理延迟反而高于原始模型。稀疏化技术在非结构化情形下能够压缩模型体积，却难以在通用硬件上直接转化为计算加速。而原生稀疏架构如 MoE 在输入分布偏斜时面临专家负载不均的挑战。线性注意力架构在长序列效率上具备显著优势，但在需要精确检索上下文的任务上，其表现弱于标准注意力，这也是当前混合线性注意力和标准注意力的根本原因。在算法层，上下文压缩与思维链压缩可能丢失关键信息而导致准确率下降，投机解码的收益高度依赖草稿模型与目标模型的输出一致性，在复杂推理任务或大批量场景下会显著衰减，提前退出思考则可能损伤长思维链任务的推理完整性。

在服务层，PD 分离架构在高并发、长输入的负载下收益显著，但在小批量或短输入场景下键值缓存的跨实例传输开销占比将显著上升，可能反而拖累端到端延迟。异构卸载通过将部分计算下放至 CPU，有效突破了单卡显存的物理上限，但跨设备传输带宽常成为吞吐瓶颈，其加速收益通常仅在稀疏激活模型（如 MoE）或离线批处理等特定场景下较为可观。综合来看，上述各项技术在真实生产部署中并不存在通用的最优解，其有效性普遍取决于工作负载特征（输入/输出长度比、并发度、服务质量目标）、模型规模与硬件条件的组合状态。

5.2 组合优化与协同关系

从全局视角看，各类推理优化技术并非彼此孤立，其实际收益不仅取决于单项技术本身，也取决于不同技术之间的协同与制约关系。总体而言，作用于不同瓶颈的技术往往具有较强的正交性，例如量化降低显存占用与传输数据量，PD 分离缓解预填充与解码阶段的资源争用，异构卸载扩展可用存储层级，三者在高并发长上下文场景下通常可形成叠加增益。此外，稀疏激活架构与 CPU-GPU 协同也具有天然互补性，因为部分专家或层可在低成本设备上异步执行。

然而，作用于同一资源瓶颈的技术则可能存在收益重叠甚至相互冲突，例如键值缓存量化与上下文压缩虽均可减小缓存开销，但前者压缩数值精度、后者删除历史词元，组合后可能带来精度损失累积；思维链压缩与投机解码虽然都能降低推理延迟，但前者改变推理过程长度与分布，后者依赖草稿模型与目标模型的一致性，两者联用时的实际收益并不稳定。因此，未来大模型推理优化不应停留于单点技术的局部改进，而需要面向具体工作负载构建一套系统级方法论。

5.3 未来方向

随着大模型应用场景持续扩展，推理优化的核心问题正在由单点性能提升转向跨层协同优化。未来，大模型推理优化将主要沿着以下 3 个方向发展。

首先，面向智能体应用的优化将成为大模型推理系统的重要发展方向。随着大模型应用从单轮对话向多轮规划、工具调用和多模态交互演进[115-117]，推理负载特征呈现出更长上下文、更复杂调用链和更强动态性的特征。在模型层，未来模型架构需要更原生地支持长程记忆、状态复用和动态计算，降低智能体长链任务中的计算开销。在算法层，优化重点将从通用的上下文压缩和解码加速，进一步转向任务感知的上下文管理、预算控制等，使推理计算能够根据任务复杂度和中间状态动态调整。在服务层，智能体任务不再是孤立请求，而是由多个模型调用、工具调用和子任务结果构成的请求图，因此需要设计面向会话级和任务图级的调度机制，以及端到端的服务质量控制。

其次，端侧推理将推动推理优化向资源受限场景扩展。目前手机、PC 和边缘设备对本地智能能力的需求不断提升，但端侧设备受到显存容量、存储带宽、功耗和散热条件的共同约束[118-119]。在模型层，端侧部署要求模型具备更强的硬件友好性，推动极低比特量化、结构化剪枝、轻量化 MoE 和混合线性架构进一步发展。在算法层，需要根据设备状态、任务难度和网络条件动态调整上下文压缩率、生成长度和推理深度。在服务层，端云协同将成为重要形态，即简单、隐私敏感或低延迟任务在端侧完成，复杂推理和长上下文任务由云端承担计算，从而在响应速度、能耗、数据隐私和算力成本之间取得平衡。

最后，安全与效率的协同优化将成为生产级推理系统的重要需求。现有推理优化方法大多关注吞吐量、延迟和显存占用，但在多租户云服务、智能体工具调用和隐私敏感应用中，安全机制同样会影响推理系统设计[120]。在模型层，量化、剪枝和稀疏化不仅需要保持任务精度，还应评估其对安全对齐、拒答能力和鲁棒性的影响。在服务层，多租户环境下的键值缓存复用、异构卸载和 PD 分离可能引入缓存污

染、数据残留和侧信道泄露等风险,因此需要构建安全感知的资源隔离、缓存生命周期管理和访问控制机制。

作者贡献声明:叶旭明负责论文撰写;周健、吴非负责论文组织架构设计并修改论文;柏澍晗补充文献资料并协助修改论文;刘源、朱国平、刘年超提出指导意见并修改论文。

参 考 文 献

[1] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]//Proc of the 31st Conf on Neural Information Processing Systems. Red Hook, NY: Curran Associates, Inc, 2017: 5998−6008

[2] Kaplan J, McCandlish S, Henighan T, et al. Scaling laws for neural language models[J]. arXiv preprint, arXiv: 2001.08361, 2020

[3] Hoffmann J, Borgeaud S, Mensch A, et al. Training compute-optimal large language models (2022)[J]. arXiv preprint, arXiv: 2203.15556, 2022

[4] Schaeffer R, Miranda B, Koyejo S. Are emergent abilities of large language models a mirage?[C]//Proc of the 37th Conf on Neural Information Processing Systems. Red Hook, NY: Curran Associates, Inc, 2023: 55565−55581

[5] Chen Xuanting, Ye Junjie, Zu Can, et al. Robustness of GPT large language models on natural language processing tasks[J]. Journal of Computer Research and Development, 2024, 61(5): 1128−1142 (in Chinese)
(陈炫婷, 叶俊杰, 祖璨, 等. GPT 系列大语言模型在自然语言处理任务中的鲁棒性[J]. 计算机研究与发展, 2024, 61(5): 1128−1142)

[6] Sun Jiankai, Zheng Chuanyang, Xie Enze, et al. A survey of reasoning with foundation models: Concepts, methodologies, and outlook[J]. ACM Computing Surveys, 2025, 57(11): 1−43

[7] Yang Zezhou, Chen Sirong, Gao Cuiyun, et al. An empirical study of retrieval-augmented code generation: Challenges and opportunities[J]. ACM Transactions on Software Engineering and Methodology, 2025, 34(7): 188: 1−188: 28

[8] Achiam J, Adler S, Agarwal S, et al. Gpt−4 technical report[J]. arXiv preprint, arXiv: 2303.08774, 2023

[9] Chen M, Tworek J, Jun H. Evaluating large language models trained on code[J]. arXiv preprint, arXiv: 2107.03374, 2021

[10] Ge Xuran, Ou Yang, Wang Bo, et al. Survey of storage optimization techniques in large language model inference[J]. Journal of Computer Research and Development, 2025, 62(3): 545−562 (in Chinese)
(葛旭冉, 欧洋, 王博, 等. 大语言模型推理中的存储优化技术综述[J]. 计算机研究与发展, 2025, 62(3): 545−562)

[11] Grattafiori A, Dubey A, Jauhri A, et al. The LLaMA 3 herd of models[J]. arXiv preprint, arXiv: 2407.21783, 2024

[12] Guo Daya, Yang Dejian, Zhang Haowei, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning[J]. Nature, 2025, 645(8081): 633−638

[13] Wang Lei, Ma Chen, Feng Xueyang, et al. A survey on large language model based autonomous agents[J]. Frontiers of Computer Science, 2024, 18(6): 186345

[14] Lin Delü, He Kun. A low-precision loss quantification method for self-attention module[J]. Journal of Computer Research and Development, 2026, 63(1): 162−175 (in Chinese)
(林德铝, 何琨. 一种自注意力模块的低精度损失量化方法[J]. 计算机研究与发展, 2026, 63(1): 162−175)

[15] Zhou Zixuan, Ning Xuefei, Hong Ke, et al. A survey on efficient inference for large language models[J]. arXiv preprint, arXiv: 2404.14294, 2024

[16] Shazeer N. Fast transformer decoding: One write-head is all you need[J]. arXiv preprint, arXiv: 1911.02150, 2019

[17] Ainslie J, Lee-Thorp J, de Jong M, et al. GQA: Training generalized multi-query transformer models from multi-head checkpoints[C]//Proc of the 28th Conf on Empirical Methods in Natural Language Processing. Stroudsburg, PA: ACL, 2023: 4895−4901

[18] Liu Aixin, Feng Bei, Wang Bin, et al. Deepseek-V2: A strong, economical, and efficient mixture-of-experts language model[J]. arXiv preprint, arXiv: 2405.04434, 2024

[19] Li Zhiyuan, Xia Tingyu, Chang Yi, et al. A survey of RWKV[J]. Neurocomputing, 2025, 649: 130711

[20] Gu A, Dao T. Mamba: Linear-time sequence modeling with selective state spaces[J]. arXiv preprint, arXiv: 2312.00752, 2023

[21] Liskavets B, Ushakov M, Roy S, et al. Prompt compression with context-aware sentence encoding for fast and improved LLM inference[C]//Proc of the 39th AAAI Conf on Artificial Intelligence. Palo Alto, CA: AAAI, 2025: 24595−24604

[22] Yan Minghao, Agarwal S, Venkataraman S. Decoding speculative decoding[C]//Proc of the 2025 Conf of the Nations of the Americas Chapter of the Association for Computational Linguistics. Stroudsburg, PA: ACL, 2025: 6460−6473

[23] Zeng Zihao, Huang Xuyao, Li Boxiu, et al. Done is better than perfect: Unlocking efficient reasoning by structured multi-turn decomposition[J]. arXiv preprint, arXiv: 2505.19788, 2025

[24] Dao T, Fu D, Ermon S, et al. FlashAttention: Fast and memory-efficient exact attention with IO-awareness[C]//Proc of the 36th Conf on Neural Information Processing Systems. Red Hook, NY: Curran Associates, Inc, 2022: 67−78

[25] Dao T, Haziza D, Massa F, Sizov G, et al. Flash-decoding for long-context inference[EB/OL]. 2023[2025-12-14]. https://crfm.stanford.edu/2023/10/12/flashdecoding.html

[26] NVIDIA. FasterTransformer[EB/OL]. 2021[2025-12-14]. https://github.com/NVIDIA/FasterTransformer

[27] NVIDIA. TensorRT-LLM: A TensorRT toolbox for optimized large language model inference[EB/OL]. 2023[2025-12-14]. https://github.com/NVIDIA/TensorRT-LLM

[28] Kwon W, Li Zhuohan, Zhuang Siyuan, et al. Efficient memory management for large language model serving with Paged-

Attention[C]//Proc of the 29th ACM SIGOPS Symp on Operating Systems Principles. New York: ACM, 2023: 611−626

[29] Zheng Lianmin, Yin Liangsheng, Xie Zhiqiang, et al. SGLang: Efficient execution of structured language model programs[C]//Proc of the 38th Conf on Neural Information Processing Systems. Red Hook, NY: Curran Associates, Inc, 2024: 62557−62583

[30] Hugging Face. Text generation inference[EB/OL]. 2023[2025-12-14]. https://github.com/huggingface/text-generation-inference

[31] GERGANOV G. llama. cpp: Port of Facebook's LLaMA model in C/C++[EB/OL]. 2023[2025-12-14]. https://github.com/ggerganov/llama.cpp

[32] Chen Hongtao, Xie Weiyu, Zhang Boxin, et al. KTransformers: Unleashing the full potential of CPU/GPU hybrid inference for MoE models[C]//Proc of the 31st ACM SIGOPS Symp on Operating Systems Principles. New York: ACM, 2025: 1014−1029

[33] Su Jianlin, Ahmed M, Lu Yu, et al. Roformer: Enhanced transformer with rotary position embedding[J]. Neurocomputing, 2024, 568: 127063

[34] Zhang Biao, Sennrich R. Root mean square layer normalization[C]//Proc of the 33rd Conf on Neural Information Processing Systems. Red Hook, NY: Curran Associates, Inc, 2019: 12381−12392

[35] Fan Wenqi, Ding Yujuan, Ning Liangbo, et al. A survey on RAG meeting LLMs: Towards retrieval-augmented large language models[C]//Proc of the 30th ACM SIGKDD Conf on Knowledge Discovery and Data Mining. New York: ACM, 2024: 6491−6501

[36] Lightman H, Kosaraju V, Burda Y, et al. Let's verify step by step[C]//Proc of the 12th Int Conf on Learning Representations. Amherst, MA: OpenReview. net, 2024: 16954−16977

[37] Dettmers T, Lewis M, Belkada Y, et al. GPT3. int8(): 8-bit matrix multiplication for transformers at scale[C]//Proc of the 36th Conf on Neural Information Processing Systems. Red Hook, NY: Curran Associates, Inc, 2022: 30318−30332

[38] Xiao Guangxuan, Lin Ji, Seznec M, et al. SmoothQuant: Accurate and efficient post-training quantization for large language models[C]//Proc of the 40th Int Conf on Machine Learning. New York: PMLR, 2023: 38087−38099

[39] Frantar E, Ashkboos S, Hoefler T, et al. OPTQ: Accurate quantization for generative pre-trained transformers[C]//Proc of the 11th Int Conf on Learning Representations. Amherst, MA: OpenReview. net, 2023: 33657−33672

[40] Lin Ji, Tang Jiaming, Tang Haotian, et al. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration[J]. GetMobile: Mobile Computing and Communications, 2024, 28(4): 12−17

[41] Ashkboos S, Mohtashami A, Croci M L, et al. Quarot: Outlier-free 4-bit inference in rotated LLMs[C]//Proc of the 38th Conf on Neural Information Processing Systems. Red Hook, NY: Curran Associates, Inc, 2024: 100213−100240

[42] Liu Zechun, Zhao Changsheng, Fedorov I, et al. SpinQuant: LLM quantization with learned rotations[C]//Proc of the 13th Int Conf on Learning Representations. Amherst, MA: OpenReview. net, 2025: 64546−64570

[43] Lin Haokun, Xu Haobo, Wu Yichen, et al. DuQuant: Distributing outliers via dual transformation makes stronger quantized LLMs[C]//Proc of the 38th Conf on Neural Information Processing Systems. Red Hook, NY: Curran Associates, Inc, 2024: 87766−87800

[44] Dettmers T, Svirschevski R A, Egiazarian V, et al. SpQR: A sparse-quantized representation for near-lossless LLM weight compression[C]//Proc of the 12th Int Conf on Learning Representations. Amherst, MA: OpenReview. net, 2024: 16293−16321

[45] Kim S, Hooper C, Gholami A, et al. SqueezeLLM: Dense-and-sparse quantization[C]//Proc of the 41st Int Conf on Machine Learning. New York: PMLR, 2024: 23901−23923

[46] Egiazarian V, Panferov A, Kuznedelev D, et al. Extreme compression of large language models via additive quantization[C]//Proc of the 41st Int Conf on Machine Learning. New York: PMLR, 2024: 12284−12303

[47] Tseng A, Chee J, Sun Qingyao, et al. QuIP# even better LLM quantization with hadamard incoherence and lattice codebooks[C]//Proc of the 41st Int Conf on Machine Learning. New York: PMLR, 2024: 48630−48656

[48] Frantar E, Alistarh D. Optimal brain compression: A framework for accurate post-training quantization and pruning[C]//Proc of the 36th Conf on Neural Information Processing Systems. Red Hook, NY: Curran Associates, Inc, 2022: 4475−4488

[49] Liu Zirui, Yuan Jiayi, Jin Hongye, et al. KIVI: A tuning-free asymmetric 2bit quantization for KV cache[C]//Proc of the 41st Int Conf on Machine Learning. New York: PMLR, 2024: 32332−32344

[50] Hooper C, Kim S, Mohammadzadeh H, et al. KVQuant: Towards 10 million context length LLM inference with KV cache quantization[C]//Proc of the 38th Conf on Neural Information Processing Systems. Red Hook, NY: Curran Associates, Inc, 2024: 1270−1303

[51] Liu Yuhan, Li Hanchen, Cheng Yihua, et al. CacheGen: KV cache compression and streaming for fast large language model serving[C]//Proc of the ACM SIGCOMM 2024 Conf. New York: ACM, 2024: 38−56

[52] Cheng Feng, Guo Cong, Wei Chiyue, et al. Ecco: Improving memory bandwidth and capacity for LLMs via entropy-aware cache compression[C]//Proc of the 52nd Annual Int Symp on Computer Architecture. New York: ACM, 2025: 793−807

[53] Zhu Xunyu, Li Jian, Liu Yong, et al. A survey on model compression for large language models[J]. Transactions of the Association for Computational Linguistics, 2024, 12: 1556−1577

[54] Frantar E, Alistarh D. SparseGPT: Massive language models can be accurately pruned in one-shot[C]//Proc of the 40th Int Conf on Machine Learning. New York: PMLR, 2023: 10323−10337.

[55] Sun Mingjie, Liu Zhuang, Bair A, et al. A simple and effective pruning approach for large language models[C]//Proc of the 12th Int Conf on Learning Representations. Amherst, MA: OpenReview. net, 2024: 4640−4662

[56] Lepikhin D, Lee H, Xu Yuanzhong, et al. GShard: Scaling giant

models with conditional computation and automatic sharding[C]// Proc of the 9th Int Conf on Learning Representations. Amherst, MA: OpenReview. net, 2021: 14225−14247

[57] Ashkboos S, Croci M L, Do Nascimento M G, et al. SliceGPT: Compress large language models by deleting rows and columns[C]// Proc of the 12th Int Conf on Learning Representations. Amherst, MA: OpenReview. net, 2024: 37227−37246

[58] Li Yiwei, Yuan Peiwen, Feng Shaoxiong, et al. Turning dust into gold: Distilling complex reasoning capabilities from LLMs by leveraging negative data[C]//Proc of the 38th AAAI Conf on Artificial Intelligence. Palo Alto, CA: AAAI, 2024: 18591−18599

[59] Du Nan, Huang Yanping, Dai A M, et al. Glam: Efficient scaling of language models with mixture-of-experts[C]//Proc of the 39th Int Conf on Machine Learning. New York: PMLR, 2022: 5547−5569

[60] Fedus W, Zoph B, Shazeer N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity[J]. Journal of Machine Learning Research, 2022, 23(1): 1−39

[61] Liu Aixin, Feng Bei, Xue Bing, et al. Deepseek-V3 technical report[J]. arXiv preprint, arXiv: 2412.19437, 2024

[62] Jiang A Q, Sablayrolles A, Roux A, et al. Mixtral of experts[J]. arXiv preprint, arXiv: 2401.04088, 2024

[63] Lu Enzhe, Jiang Zhejun, Liu Jingyuan, et al. MoBA: Mixture of block attention for long-context LLMs[J]. arXiv preprint, arXiv: 2502.13189, 2025

[64] Piękos P, Csordás R, Schmidhuber J. Mixture of sparse attention: Content-based learnable sparse attention via expert-choice routing[J]. arXiv preprint, arXiv: 2505.00315, 2025

[65] Yuan Jingyang, Gao Huazuo, Dai Damai, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention[C]//Proc of the 63rd Annual Meeting of the Association for Computational Linguistics. Stroudsburg, PA: ACL, 2025: 23078−23097

[66] Raposo D, Ritter S, Richards B, et al. Mixture-of-depths: Dynamically allocating compute in transformer-based language models[J]. arXiv preprint, arXiv: 2404.02258, 2024

[67] Lahoti A, Li K Y, Chen B, et al. Mamba−3: Improved sequence modeling using state space principles[J]. arXiv preprint, arXiv: 2603.15569, 2026

[68] Dao T, Gu A. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality[C]//Proc of the 41st Int Conf on Machine Learning. New York: PMLR, 2024: 10041−10071

[69] Munkhdalai T, Faruqui M, Gopal S. Leave no context behind: Efficient infinite context transformers with infini-attention[J]. arXiv preprint, arXiv: 2404.07143, 2024

[70] Chen Aili, Li Aonian, Gong Bangwei, et al. Minimax-M1: Scaling test-time compute efficiently with lightning attention[J]. arXiv preprint, arXiv: 2506.13585, 2025

[71] Yang An, Li Anfeng, Yang Baosong, et al. Qwen3 technical report[J]. arXiv preprint, arXiv: 2505.09388, 2025

[72] Zhang Yu, Lin Zongyu, Yao Xingcheng, et al. Kimi linear: An expressive, efficient attention architecture[J]. arXiv preprint, arXiv: 2510.26692, 2025

[73] Liu Hao, Zaharia M, Abbeel P. RingAttention with blockwise transformers for near-infinite context[C]//Proc of the 12th Int Conf on Learning Representations. Amherst, MA: OpenReview. net, 2024: 3992−4008

[74] Yin Junqi, Palash M, Shankar M, et al. RingX: Scalable parallel attention for long-context learning on HPC[C]//Proc of Int Conf for High Performance Computing, Networking, Storage and Analysis. New York: ACM, 2025: 1395−1408

[75] Dong Qingxiu, Li Lei, Dai Damai, et al. A survey on in-context learning[C]//Proc of the 29th Conf on Empirical Methods in Natural Language Processing. Stroudsburg, PA: ACL, 2024: 1107−1128

[76] Anthropic. Introducing advanced tool use on the claude developer platform[EB/OL]. 2025[2025-12-14]. https://www.anthropic.com/engineering/advanced-tool-use

[77] Li Yuhong, Huang Yingbing, Yang Bowen, et al. SnapKV: LLM knows what you are looking for before generation[C]//Proc of the 37th Conf on Neural Information Processing Systems. Red Hook, NY: Curran Associates, Inc, 2024: 22947−22970

[78] Li Yucheng, Dong Bo, Guerin F, et al. Compressing context to enhance inference efficiency of large language models[C]//Proc of the 28th Conf on Empirical Methods in Natural Language Processing. Stroudsburg, PA: ACL, 2023: 6342−6353

[79] Jiang Huiqiang, Wu Qianhui, Lin C Y, et al. LLMLingua: Compressing prompts for accelerated inference of large language models[C]//Proc of the 28th Conf on Empirical Methods in Natural Language Processing. Stroudsburg, PA: ACL, 2023: 13358−13376

[80] Jiang Huiqiang, Wu Qianhui, Luo Xufang, et al. Longllmlingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression[C]//Proc of the 62nd Annual Meeting of the Association for Computational Linguistics. Stroudsburg, PA: ACL, 2024: 1658−1677

[81] Xu Fangyuan, Shi Weijia, Choi E. RECOMP: Improving retrieval-augmented lms with context compression and selective augmentation[C]//Proc of the 12th Int Conf on Learning Representations. Amherst, MA: OpenReview. net, 2024: 55471−55495

[82] Xiao Guangxuan, Tian Yuandong, Chen Beidi, et al. Efficient streaming language models with attention sinks[C]//Proc of the 12th Int Conf on Learning Representations. Amherst, MA: OpenReview. net, 2024: 16082−16102

[83] Zhang Zhenyu, Sheng Ying, Zhou Tianyi, et al. H2O: Heavy-hitter oracle for efficient generative inference of large language models[C]//Proc of the 37th Conf on Neural Information Processing Systems. Red Hook, NY: Curran Associates, Inc, 2023: 34661−34710

[84] Ge Suyu, Zhang Yunan, Liu Liyuan, et al. Model tells you what to discard: Adaptive KV cache compression for LLMs[C]//Proc of the 12th Int Conf on Learning Representations. Amherst, MA: OpenReview. net, 2024: 43023−43036

[85] Cai Zefan, Zhang Yichi, Gao Bofei, et al. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling[J]. arXiv preprint, arXiv: 2406.02069, 2024

[86] Lee W, Lee J, Seo J, et al. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management[C]//

Proc of the 18th USENIX Symp on Operating Systems Design and Implementation. Berkeley, CA: USENIX Association, 2024: 155−172

[87] Chen Weijian, He Shuibing, Qu Haoyang, et al. IMPRESS: An importance-informed multi-tier prefix KV storage system for large language model inference[C]//Proc of the 23rd USENIX Conf on File and Storage Technologies. Berkeley, CA: USENIX Association, 2025: 187−201

[88] Xia Heming, Ge Tao, Wang Peiyi, et al. Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation[C]//Proc of the Findings of the 28th Conf on Empirical Methods in Natural Language Processing. Stroudsburg, PA: ACL, 2023: 3909−3925

[89] Li Yuhui, Wei Fangyun, Zhang Chao, et al. EAGLE−3: Scaling up inference acceleration of large language models via training-time test[J]. arXiv preprint, arXiv: 2503.01840, 2025

[90] Miao Xupeng, Oliaro G, Zhang Zhihao, et al. SpecInfer: Accelerating large language model serving with tree-based speculative inference and verification[C]//Proc of the 29th ACM Int Conf on Architectural Support for Programming Languages and Operating Systems. New York: ACM, 2024: 932−949

[91] Cai Tianle, Li Yuhong, Geng Zhengyang, et al. MEDUSA: Simple LLM inference acceleration framework with multiple decoding heads[C]//Proc of the 41st Int Conf on Machine Learning. New York: PMLR, 2024: 5209−5235

[92] He Zhenyu, Zhong Zexuan, Cai Tianle, et al. REST: Retrieval-based speculative decoding[C]//Proc of the 2024 Conf of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. Stroudsburg, PA: ACL, 2024: 1582−1595

[93] Xia Yu, Wang Rui, Liu Xu, et al. Beyond chain-of-thought: A survey of chain-of-x paradigms for LLMs[C]//Proc of the 31st Int Conf on Computational Linguistics. Stroudsburg, PA: ACL, 2025: 10795−10809

[94] Chen Xingyu, Xu Jiahao, Liang Tian, et al. Do not think that much for 2+ 3=? on the overthinking of o1-like LLMs[J]. arXiv preprint, arXiv: 2412.21187, 2024

[95] Yang Chenxu, Si Qingyi, Duan Yongjie, et al. Dynamic early exit in reasoning models[J]. arXiv preprint, arXiv: 2504.15895, 2025

[96] Muennighoff N, Yang Zitong, Shi Weijia, et al. s1: Simple test-time scaling[C]//Proc of the Findings of the 30th Conf on Empirical Methods in Natural Language Processing. Stroudsburg, PA: ACL, 2025: 20286−20332

[97] Hou Bairu, Zhang Yang, Ji Jiabao, et al. ThinkPrune: Pruning long chain-of-thought of LLMs via reinforcement learning[J]. arXiv preprint, arXiv: 2504.01296, 2025

[98] Dao T. FlashAttention−2: Faster attention with better parallelism and work partitioning[C]//Proc of the 12th Int Conf on Learning Representations. Amherst, MA: OpenReview. net, 2024: 26738−26751

[99] Shah J, Bikshandi G, Zhang Ying, et al. Flashattention−3: Fast and accurate attention with asynchrony and low-precision[C]//Proc of the 38th Conf on Neural Information Processing Systems. Red Hook, NY: Curran Associates, Inc, 2024: 68658−68685

[100] Hong Ke, Dai Guohao, Xu Jiaming, et al. FlashDecoding++: Faster large language model inference on GPUs[J]. arXiv preprint, arXiv: 2311.01282, 2023

[101] Qin Ruoyu, Li Zheming, He Weiran, et al. Mooncake: Trading more storage for less computation: A KVCache-centric architecture for serving LLM chatbot[C]//Proc of the 23rd USENIX Conf on File and Storage Technologies. Berkeley, CA: USENIX Association, 2025: 155−170

[102] Yu G I, Jeong J S, Kim G W, et al. Orca: A distributed serving system for Transformer-Based generative models[C]//Proc of the 16th USENIX Symp on Operating Systems Design and Implementation. Berkeley, CA: USENIX Association, 2022: 521−538

[103] Agrawal A, Kedia N, Panwar A, et al. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve[C]//Proc of the 18th USENIX Symp on Operating Systems Design and Implementation. Berkeley, CA: USENIX Association, 2024: 117−134

[104] Zhong Yinmin, Liu Shengyu, Chen Junda, et al. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving[C]//Proc of the 18th USENIX Symp on Operating Systems Design and Implementation. Berkeley, CA: USENIX Association, 2024: 193−210

[105] Patel P, Choukse E, Zhang Chaojie, et al. Splitwise: Efficient generative LLM inference using phase splitting[C]//Proc of the 51st Annual Int Symp on Computer Architecture. Piscataway, NJ: IEEE, 2024: 118−132

[106] Hu Cunchen, Huang Heyang, Xu Liangliang, et al. ShuffleInfer: Disaggregate LLM inference for mixed downstream workloads[J]. ACM Trans on Architecture and Code Optimization, 2025, 22(2): 77: 1−77: 24

[107] Jin Yibo, Wang Tao, Lin Huimin, et al. P/D-Serve: Serving disaggregated large language model at scale[J]. arXiv preprint, arXiv: 2408.08147, 2024

[108] Liao Junhan, Xu Minxian, Zheng Wanyi, et al. DOPD: A dynamic PD-disaggregation architecture for maximizing goodput in LLM inference serving[J]. IEEE Transaction on Services Computing, 2026, 19(2): 1134−1147

[109] Aminabadi R Y, Rajbhandari S, Awan A A, et al. DeepSpeed-inference: Enabling efficient inference of transformer models at unprecedented scale[C]//Proc of Int Conf for High Performance Computing, Networking, Storage and Analysis. Piscataway, NJ: IEEE, 2022: 46: 1−46: 15

[110] Gao Bin, He Zhuomin, Sharma P, et al. Cost-efficient large language model serving for multi-turn conversations with CachedAttention[C]//Proc of the 2024 USENIX Annual Technical Conf. Berkeley, CA: USENIX Association, 2024: 111−126

[111] Jeong J, Ahn J. Accelerating LLM serving for multi-turn dialogues with efficient resource management[C]//Proc of the 30th ACM Int Conf on Architectural Support for Programming Languages and Operating Systems, Volume 2. New York: ACM, 2025: 1−15

[112] Gao Shiwei, Chen Youmin, Shu Jiwu. Fast state restoration in LLM serving with HCache[C]//Proc of the 20th European Conf on Computer Systems. New York: ACM, 2025: 128−143

[113] Wu Yongtong, Chen Shaoyuan, Zhong Yinmin, et al. DualPath: Breaking the storage bandwidth bottleneck in agentic LLM inference[J]. arXiv preprint, arXiv: 2602.21548, 2026

[114] Liu Yuhan, Cheng Yihua, Yao Jiayi, et al. LMCache: An efficient KV cache layer for enterprise-scale LLM inference[J]. arXiv preprint, arXiv: 2510.09665, 2025

[115] Zhang Chi, Yang Zhao, Liu Jiaxuan, et al. AppAgent: Multimodal agents as smartphone users[C]//Proc of the 2025 CHI Conf on Human Factors in Computing Systems. New York: ACM, 2025: 70: 1−70: 20

[116] Li Xinzhe. A review of prominent paradigms for LLM-based agents: Tool use, planning (including RAG), and feedback learning[C]//Proc of the 31st Int Conf on Computational Linguistics. Stroudsburg, PA: ACL, 2025: 9760−9779

[117] Shi Haojun, Ye Suyu, Fang Xinyu, et al. MuMA-ToM: Multi-modal multi-agent theory of mind[C]//Proc of the 39th AAAI Conf on Artificial Intelligence. Palo Alto, CA: AAAI, 2025: 1510−1519

[118] Husom E J, Goknil A, Astekin M, et al. Sustainable LLM inference for edge AI: Evaluating quantized LLMs for energy efficiency, output accuracy, and inference latency[J]. ACM Transactions on Internet of Things, 2025, 6(4): 1−35

[119] Qin Ruiyang, Liu Dancheng, Xu Chenhui, et al. Empirical guidelines for deploying LLMs onto resource-constrained edge devices[J]. ACM Transactions on Design Automation of Electronic Systems, 2025, 30(5): 1−58

[120] He Feng, Zhu Tianqing, Ye Dayong, et al. The emerged security and privacy of LLM agent: A survey with case studies[J]. ACM Computing Surveys, 2025, 58(6): 1−36



Ye Xuming, born in 2000. PhD candidate. His main research interests include LLM inference acceleration and storage systems.

叶旭明, 2000 年生。博士研究生。主要研究方向为大模型推理加速、存储系统。



Zhou Jian, born in 1986. PhD, associate professor, PhD supervisor. His main research interests include computer architecture, disaggregated storage systems, and intelligent storage.

周　健, 1986 年生。博士, 副教授, 博士生导师。主要研究方向为计算机体系结构、分离式存储系统、智能存储。



Bai Shuhan, born in 1996. PhD, postdoctoral researcher. Her main research interests include storage systems, processing near memory, and intelligent data management.

柏澍晗, 1996 年生。博士, 博士后。主要研究方向为存储系统、近存计算、智能数据管理。



Liu Yuan, born in 1990. PhD candidate. His main research interests include storage systems and AI systems.

刘　源, 1990 年生。博士研究生。主要研究方向为存储系统、人工智能系统。



Zhu Guoping, born in 1974. Bachelor, professorate senior engineer. His research interests include distributed software systems and hardware-software co-design for AI infrastructure.

朱国平, 1974 年生。学士, 正高级工程师。主要研究方向为分布式软件系统、AI 基础设施软硬件协同技术。



Liu Nianchao, born in 1980. Bachelor, senior engineer. His research interests include hardware-software co-design for AI infrastructure and memory pooling technology for SuperPod architectures.

刘年超, 1980 年生。学士, 高级工程师。主要研究方向为 AI 基础设施软硬件协同、超节点内存池化技术。



Wu Fei, born in 1975. PhD, professor, PhD supervisor. Senior member of CCF, member of IEEE and ACM. Her main research interests include computer architecture, non-volatile memory, and intelligent storage.

吴　非, 1975 年生。博士, 教授, 博士生导师。CCF 高级会员, IEEE 和 ACM 会员。主要研究方向为计算机体系结构、非易失性存储器、智能存储。